

Computer Graphics

(CSP 371)

PREPARE BY:

Dr. Hala Elhadidy

2026

Content

Chapters	Names	Pages
Chapter 1	Introduction to Computer Graphics (CG)	3
Chapter 2	Graphics Techniques	17
Chapter 3	Geometrical Transformations	56
Chapter 4	Practical	83

Chapter 1

Introduction to Computer Graphics (CG)

- 1. Definitions and terminologies**
- 2. Applications of Computer Graphics**
- 3. Graphics Package**
 - 3.1 Pixels and The Frame Buffer**
 - 3.2 Input Devices**
 - 3.3 OUTPUT DEVICES**
 - 3.4 CPU and GPU**
- 4. Interactive Computer Graphics**
- 5. The Graphics Pipeline**
- 6. Efficiency**
- 7. Designing and Coding Graphics Programs**

Chapter 1

Introduction to Computer Graphics

1.1 Definitions and terminologies:

The term computer graphics (CG) describes the use of computers to create and manipulate images. Graphics can be two- or three-dimensional. Computer Graphics is principally concerned with the generation of images, with wide ranging applications from entertainment to scientific visualization. In other words, we can say that computer graphics is a rendering tool for the generation and manipulation of images. Thus, by using a computer as a rendering tool for the generation and manipulation of images is called computer graphics. Computer graphics is an art of drawing pictures on computer screens with the help of programming. It involves computations, creation, and manipulation of data. Graphics is a vast field that encompasses almost any graphical aspect like special effects, simulation and training, games, medical imagery, animations and much more. *“Computer Graphics refers to any sketch, drawing, special artwork or other material generated with the help of computer to pictorially depict an object or a process or convey information, as a supplement to or instead of written descriptions”*. It relies on an internal model of the scene, that is, a mathematical representation suitable for graphical computations. The model describes the 3D shapes, layout, projections to compute a 2D image from a given viewpoint and rendering which involves projecting the objects (perspective), handling visibility (which parts of objects are hidden) and computing their appearance and lighting interactions and materials of the scene. Computer Graphics is the creation and manipulation of images or pictures with the help of computers.

There are two types of computer graphics:

- 1) Passive Computer Graphics (Non-interactive Computer Graphics)
- 2) Active Computer Graphics (Interactive Computer Graphics)

The major product of computer graphics is a picture. With the help of CG, pictures can be represented in 2D and 3D space. Many applications show various parts of the displayed

picture changing in size and orientation. Such type of transformations i.e. the pictures can be made to grow, shrink, rotate and etc. can be achieved through CG. The display on them is often too big to be shown in their entirety. Thus, with the help of CG, a technique called clipping can be used to select just those parts of the picture that lie on the screen and to discard the rest.

1.2 Applications of Computer Graphics

The core elements of computer graphics include the following:

- ✓ Modeling: Involves representation of choices, geometric processing
- ✓ Rendering: Executes geometric transformation, visibility, implements simulation of light
- ✓ Interaction: Synergy of input/output devices, tools
- ✓ Animation: Simulation of lifelike characters, natural phenomena, their interactions, surrounding environments
- ✓ CAD/CAM: stands for computer-aided design and computer-aided manufacturing. These fields use computer technology to design parts and products on the computer and then, using these virtual designs, to guide the manufacturing process. For example, many mechanical parts are designed in a 3D computer modeling package and then automatically produced on a computer-controlled milling device.
- ✓ Simulation: can be thought of as accurate video gaming. For example, a flight simulator uses sophisticated 3D graphics to simulate the experience of flying an airplane. Such simulations can be extremely useful for initial training in safety-critical domains such as driving, and for scenario training for experienced users

such as specific fire-fighting situations that are too costly or dangerous to create physically.

- ✓ Medical imaging creates meaningful images of scanned patient data. For example, a computed tomography (CT) dataset is composed of a large 3D rectangular array of density values. Computer graphics is used to create shaded images that help doctors extract the most salient information from such data.

Computer graphics can be broadly divided into the following classes:

- ✓ Business Presentation Graphics, which refers to graphics, such as bar-charts, histograms, pie-charts, pictograms, x-y charts, etc.
- ✓ Scientific Graphics, such as x-y plots, curve-fitting, contour plots, system or program flowcharts etc.
- ✓ Cartography (Scaled Drawings), such as architectural representations, drawings of buildings, bridges, and machines.
- ✓ Satellite Imaging: Geodesic images.
- ✓ Photo Enhancement: Sharpening blurred photos.
- ✓ Medical imaging – MRIs, CAT scans, etc.: Non-invasive internal examination.
- ✓ Engineering drawings: mechanical, electrical, civil, Replacing the blueprints of the past.
- ✓ Typography: The use of character images in publishing – replacing the hard type of the past.
- ✓ Cartoons and artwork, including advertisements.
- ✓ Simulation and modeling: Replacing physical modeling and enactments
- ✓ Graphics User Interfaces (GUIs) A graphic, mouse-oriented paradigm which allows the user to interact with a computer. The images that appear and are designed to help the user utilize the software without having to refer to manuals or read a lot of text on the monitor.

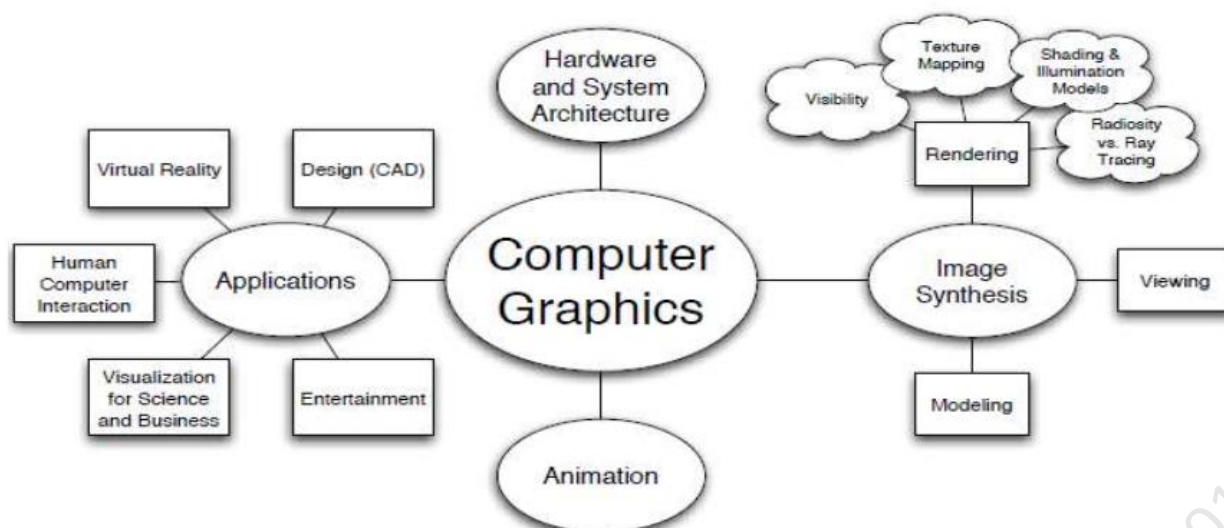


Figure 1 Some Application Areas of Computer Graphics

1.3 Graphics Package

A computer graphics system is a computer system which must have all the components of a general-purpose computer system. Considering the high-level view of a graphics system, there are six major elements in the Graphics system:

1. Input devices
2. Central Processing Unit
3. Graphics Processing Unit
4. Memory
5. Frame buffer
6. Output devices

This model is general enough to include workstations and personal computers, interactive game systems, mobile phones, GPS systems, and sophisticated image generation systems. Although most of the components are present in a standard computer, it is the way each element is specialized for computer graphics that characterizes this diagram as a portrait of a graphics system.

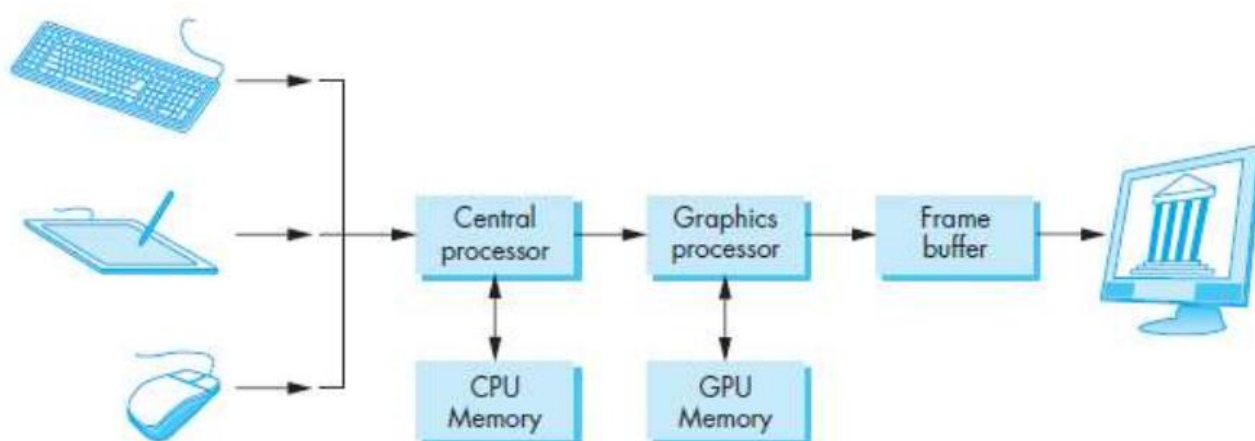


Figure 2 Graphics System

1.3.1 Pixels and The Frame Buffer

Virtually all modern graphics systems are raster based. The image we see on the output device is an array—the **raster**—of picture elements, or **pixels**, produced by the graphics system. An image that is presented on the computer screen is made up of **pixels**. The screen consists of a rectangular grid of pixels, arranged in rows and columns. The pixels are small enough that they are not easy to see individually. At a given time, each pixel can show only one color. Most screens these days use 24-bit color, where a color can be specified by three 8-bit numbers, giving the levels of red, green, and blue in the color. Any color that can be shown on the screen is made up of some combination of these three “primary” colors. Other formats are possible, such as grayscale, where each pixel is some shade of gray and the pixel color is given by one number that specifies the level of gray on a black-to-white scale. Typically, 256 shades of gray are used. Early computer screens used indexed color, where only a small set of colors, usually 16 or 256, could be displayed.

The color values for all the pixels on the screen are stored in a large block of memory known as a **frame buffer**. Changing the image on the screen requires changing color values that are stored in the frame buffer. The screen is redrawn many times per

second, so that almost immediately after the color values are changed in the frame buffer, the colors of the pixels on the screen will be changed to match, and the displayed image will change. Its **resolution**—the number of pixels in the frame buffer—determines the detail that you can see in the image. The **depth**, or **precision**, of the frame buffer, defined as the number of bits that are used for each pixel, determines properties such as how many colors can be represented on a given system. For example, a 1-bit-deep frame buffer allows only two colors, whereas an 8-bit-deep frame buffer allows 28 (256) colors. In **full-color** systems, there are 24 (or more) bits per pixel. Such systems can display sufficient colors to represent most images realistically. They are also called **true-color** systems, or **RGB-color** systems, because individual groups of bits in each pixel are assigned to each of the three primary colors—red, green, and blue—used in most displays. **High dynamic range** (HDR) systems use 12 or more bits for each color component. Until recently, frame buffers stored colors in integer formats. Recent frame buffers use floating point and thus support HDR colors more easily.

Resolution	Number of Pixels	Aspect Ratio
320*200	64000	8:5
640*480	307200	4:3
800*600	480000	4:3
1024*768	786432	4:3
1280*1024	1310720	5:4
1600*1200	1920000	4:3

Figure 3 The Relation between Resolution, Pixels and Aspect Ratio of a Graphical Device

1.3.2 Input Devices

Most graphics systems provide a keyboard and at least one other input device. The most common input devices are the mouse, the joystick, and the data tablet. Each provides positional information to the system, and each usually is equipped with one or more buttons to provide signals to the processor. Often called **pointing devices**, these devices allow a user to indicate a particular location on the display. Some commercial input devices used in Graphics System are:

- 2D mice
- Scanners
- Light Pens
- Digitizers
- Digital Camera
- Video Camera
- Touch-sensitive screens
- Joysticks
- Trackballs
- Thumb wheels
- Microphones (voice data entry)
- Touch Panels

1.3.3 Output Devices

One of the common physical output devices for display (or **monitor**) was the **cathode-ray tube (CRT)**. A simplified picture of a CRT is shown in figure below which shows the working principle of CRT. When electrons strike the phosphor coating on the tube, light is emitted. The direction of the beam is controlled by two pairs of deflection plates. The output of the computer is converted, by digital to-analog converters, to

voltages across the x and y deflection plates. Light appears on the surface of the CRT when a sufficiently intense beam of electrons is directed at the phosphor.

1.3.4 CPU and GPU

In a simple system, there may be only one processor, the **central processing unit (CPU)** of the system, which must do both the normal processing and the graphical processing. The main graphical function of the processor is to take specifications of graphical primitives (**such as lines, circles, and polygons**) generated by application programs and to assign values to the pixels in the frame buffer that best represent these entities. For example, a triangle is specified by its three vertices, but to display its outline by the three line segments connecting the vertices, the graphics system must generate a set of pixels that appear as line segments to the viewer. The conversion of geometric entities to pixel colors and locations in the frame buffer is known as **rasterization**, or **scan conversion**. In early graphics systems, the frame buffer was part of the standard memory that could be directly addressed by the CPU.

Today, virtually all graphics systems are characterized by special-purpose **graphics processing units (GPUs)**, to carry out specific graphics functions. The GPU can be either on the mother board of the system or on a graphics card. The frame buffer is accessed through the graphics processing unit and usually is on the same circuit board as the GPU. GPUs are characterized by both special-purpose modules geared toward graphical operations and a high degree of parallelism—recent GPUs contain over 100 processing units, each of which is user programmable. GPUs are so powerful that they can often be used as mini supercomputers for general purpose computing.

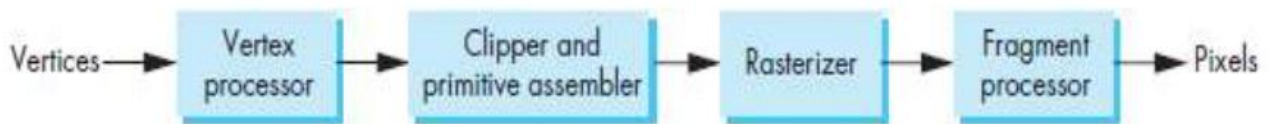


Figure 4: The Graphic Processor Architecture

1. 4 Interactive Computer Graphics

The Interactive computer graphics (ICG) provides two way communications between the computer and the user. The various applications of ICG are as follows. Using ICG system the integrated electronic circuits which are very complex can be drawn in a much shorter time. It is very useful in training of the pilots as they spend much of training on the ground at the controls of a flight simulator and not in a real aircraft. There are many tasks that can be made easier & less expensive by the use of ICG. The effectiveness of the ICG is the speed with which the user can absorb the displayed information. The Interactive Graphics display consists of three major components (1) Frame Buffer (2) T.V. Monitor (3) Display Controller as follows & shown in Figure 5:

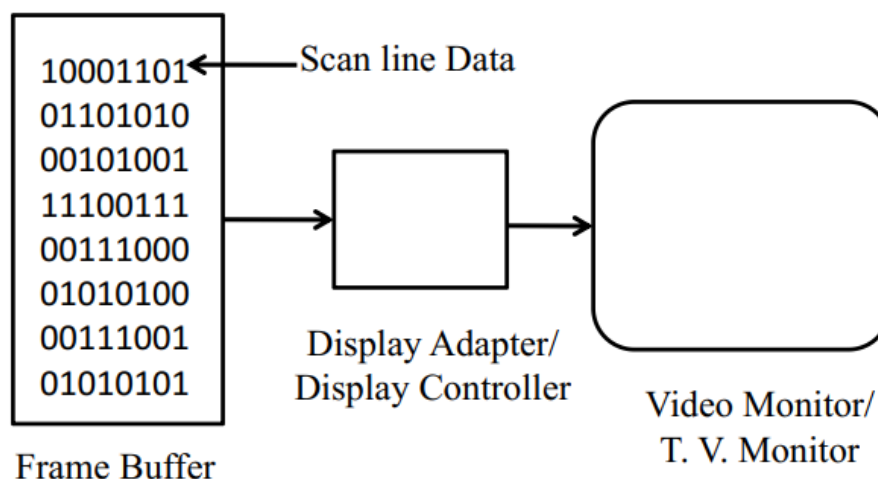


Figure 5 Interactive Computer Graphics

A. Frame Buffer

The images that are to be displayed are stored in a frame buffer in the form of matrix of intensity values. The frame buffer contains the image stored in binary form as a matrix of 0's

and 1's which represent the pixel. 0 indicates the darkness and 1 indicates the image. The **Frame Buffer** holds the set of intensity values for all the **screen points**. The intensity values stored in a Frame Buffer are retrieved and painted on a screen one row at a time. This row is called as **scan line**.

B. Display Controller

The Display Controller passes the contents of frame buffer to the T.V. Monitor. Display Controller reads successive bytes of data from the frame buffer & then converts 0's and 1's into the corresponding video signal. These signals are fed to the T.V. Monitor.

C. T.V. Monitor

The T.V. Monitor then produces black and white pattern on the screen. The frame Buffer contents are to be modified, in order to represent the new pattern of pixels or if some changes are to be made on the displayed picture.

1.5. The Graphics Pipeline

Figure 6 The Pipeline Architecture of Graphics Package. The functioning of a standard graphics system is typically described by an abstraction called the **graphics pipeline**. The term “pipeline” is used because the transformation from mathematical model to pixels on the screen involves multiple steps, and in a typical architecture, these are performed in sequence; the results of one stage are pushed on to the next stage so that the first stage can begin processing the next polygon immediately.

Each object comprises a set of graphical primitives. Each primitive comprises a set of vertices. We can think of the collection of primitive types and vertices as defining the **geometry** of the scene. In a complex scene, there may be thousands—even millions—of vertices that define the objects. We must process all these vertices in a

similar manner to form an image in the frame buffer. If we think in terms of processing the geometry of our objects to obtain an image, we can employ the four major steps in the imaging process:

1. Vertex processing
2. Clipping and primitive assembly
3. Rasterization
4. Fragment processing

No matter with which advance graphic software you are working with, if your output device or Graphics hardware is not good, or hardware handling that software is not good, then ultimate result will be not good, as discussed in the previous section to work with graphic packages.

1.6 Efficiency

Efficiency is achieved through careful tradeoffs. However, programmers should pay more attention to memory access patterns than to operation counts. Since that trend continues, the importance of limited and coherent memory access for optimization should only increase. A reasonable approach to making code fast is to proceed in the following order, taking only those steps which are needed:

1. Write the code in the most straightforward way possible. Compute intermediate results as needed on the fly rather than storing them.
2. Compile in optimized mode.
3. Use any profiling tools exist to find critical bottlenecks.
4. Examine data structures to look for ways to improve locality. If possible, make data unit sizes match the cache/page size on the target architecture.
5. If profiling reveals bottlenecks in numeric computations, examine the assembly code generated by the compiler for missed efficiencies. Rewrite source code to solve any problems you find.

The most important of these steps is the first one. Most “optimizations” make the code harder to read without speeding things up. In addition, time spent upfront optimizing code is usually better spent correcting bugs or adding features. Also, beware of suggestions from old texts; some classic tricks such as using integers instead of reals may no longer yield speed because modern CPUs can usually perform floating-point operations just as fast as they perform integer operations. In all situations, profiling is needed to be sure of the merit of any optimization for a specific machine and compiler.

1.7 Designing and Coding Graphics Programs

Certain common strategies are often useful in graphics programming. In this section we provide some advice that you may find helpful as you implement the methods you learn about in this book.

1.7.1 Class Design

A key part of any graphics program is to have good classes or routines for geometric entities such as vectors and matrices, as well as graphics entities such as RGB colors and images. These routines should be made as clean and efficient as possible. A universal design question is whether locations and displacements should be separate classes because they have different operations, e.g., a location multiplied by one-half makes no geometric sense while one-half of a displacement does. There is little agreement on this question, which can spur hours of heated debate among graphics practitioners, but for the sake of example let’s assume we will not make the distinction. This implies that some basic classes to be written include:

- **vector2**. A 2D vector class that stores an x - and y -component. It should store these components in a length-2 array so that an indexing operator can be well supported. You should also include operations for vector addition, vector subtraction, dot product, cross product, scalar multiplication, and scalar division.
- **vector3**. A 3D vector class analogous to vector2.
- **hvector**. A homogeneous vector with four components.

- **RGB.** An RGB color that stores three components. You should also include operations for RGB addition, RGB subtraction, RGB multiplication, scalar multiplication, and scalar division.
- **transform.** A 4×4 matrix for transformations. You should include a matrix multiply and member functions to apply to locations, directions, and surface normal vectors.
- **image.** A 2D array of RGB pixels with an output operation.

1.7.2 Float vs. Double

Modern architecture suggests that keeping memory use down and maintaining coherent memory access are the keys to efficiency. This suggests using single precision data. However, avoiding numerical problems suggests using double precision arithmetic.

1.7.3 Debugging Graphics Programs

If you ask around, you may find that as programmers become more experienced, they use traditional debuggers less and less. One reason for this is that using such debuggers is more awkward for complex programs than for simple programs. Another reason is that the most difficult errors are conceptual ones where the wrong thing is being implemented, and it is easy to waste large amounts of time stepping through variable values without detecting such cases.

The Scientific Method

In graphics programs there is an alternative to traditional debugging that is often very useful. We create an image and observe what is wrong with it. Then, we develop a hypothesis about what is causing the problem and test it. For example, in a ray-tracing program we might have many somewhat random looking dark pixels. This is the classic “shadow acne” problem that most people run into when they write a ray tracer. Traditional debugging is not helpful here; instead, we must realize that the shadow rays are hitting the surface being shaded. We might notice that the color of the dark spots is the ambient color, so the direct lighting is what is missing.

Direct lighting can be turned off in shadow, so you might hypothesize that these points are incorrectly being tagged as in shadow when they are not. To test this hypothesis, we could turn off the shadowing check and recompile. This would indicate that these are false shadow tests, and we could continue our detective work. The key reason that this method can sometimes be good practice is that we never had to spot a false value or really determine our conceptual error. Instead, we just narrowed in on our conceptual error experimentally. Typically, only a few trials are needed to track things down, and this type of debugging is enjoyable.

Images as Coded Debugging Output

In many cases, the easiest channel by which to get debugging information out of a graphics program is the output image itself. If you want to know the value of some variable for part of a computation that runs for every pixel, you can just modify your program temporarily to copy that value directly to the output image and skip the rest of the calculations that would normally be done. For instance, if you suspect a problem with surface normals is causing a problem with shading, you can copy the normal vectors directly to the image (x goes to red, y goes to green, z goes to blue), resulting in a color-coded illustration of the vectors actually being used in your computation. Or, if you suspect a particular value is sometimes out of its valid range, make your program write bright red pixels where that happens.

Other common tricks include drawing the back sides of surfaces with an obvious color, coloring the image by the ID numbers of the objects, or coloring pixels by the amount of work they took to compute.

Using a Debugger

There are still cases, particularly when the scientific method seems to have led to a contradiction, when there's no substitute for observing exactly what is going on. The trouble is that graphics programs often involve many, many executions of the same code,

making it completely impractical to step through in the debugger from the start. And the most difficult bugs usually only occur for complicated inputs.

A useful approach is to “**set a trap**” for the bug. First, make sure your program is deterministic—run it in a single thread and make sure that all random numbers are computed from fixed seeds. Then, find out which pixel or triangle is exhibiting the bug and add a statement before the code you suspect is incorrect that will be executed only for the suspect case. For instance, if you find that pixel (126, 247) exhibits the bug, then add:

```
if x = 126 and y = 247 then  
    print “blarg!”
```

If you set a breakpoint on the print statement, you can drop into the debugger just before the pixel you’re interested in is computed. Some debuggers have a “conditional breakpoint” feature that can achieve the same thing without modifying the code.

In the cases where the program crashes, a traditional debugger is useful for pinpointing the site of the crash. You should then start backtracking in the program, using asserts and recompiles, to find where the program went wrong. This again means the traditional step-through process is avoided, because that would not be adding the valuable asserts to your program.

Data Visualization for Debugging

Often it is hard to understand what your program is doing, because it computes a lot of intermediate results before it finally goes wrong. The situation is similar to a scientific experiment that measures a lot of data, and one solution is the same: make good plots and illustrations for yourself to understand what the data means. For instance, in a ray tracer you might write code to visualize ray trees so you can see what paths contributed to a pixel, or in an image resampling routine you might make plots that show all the points where samples are being taken from the input. Time spent writing code to visualize your program’s internal state is also repaid in a better understanding of its behavior when it comes time to optimize it.

Exercises

Part A: Answer the following questions

- 1) Discuss the application of computer graphics in entertainment.
- 2) Discuss the application of computer graphics in visualization.
- 3) What is interactive computer Graphics?
- 4) Define persistence, Resolution, Aspect Ratio, Frame Buffer.
- 5) State the function of frames buffer.

Part B – True or False

1. Computer graphics is only used in entertainment applications such as games and movies.
2. Geometry provides the shapes used in computer graphics, while algebra provides the language to express them.
3. The graphics pipeline stages are all executed on the CPU.
4. Real numbers can always be represented exactly in computer memory.
5. Ray tracing uses a pipeline structure similar to real-time rendering.
6. Anti-aliasing improves image quality by smoothing jagged edges.
7. Vectors in 3D graphics represent directions and magnitudes in space.
8. Points and vectors have the same geometric meaning and mathematical operations.
9. A quaternion can represent rotation in 3D without gimbal lock.
10. The Babylon.js engine is a full game engine including physics and AI subsystems.
11. The GPU is specialized for highly parallel mathematical operations.
12. In a triangle mesh, each triangle is typically stored independently in memory.
13. The real-time graphics pipeline must produce frames at least 25 times per second to appear smooth.
14. Matrices can be used to represent both geometric transformations and image data.
15. The alpha channel in an image represents its brightness.
16. Ray tracing computes light interactions recursively for each ray.
17. Aliasing occurs due to continuous geometry being sampled onto a discrete pixel grid.
18. Unity's DOTS architecture is based on Data-Oriented Programming principles.
19. Real numbers in CG are mainly used for integer coordinate calculations.
20. A video is a sequence of still images shown at a high enough rate to simulate motion.

Part C – Multiple Choice Questions

- 1- The main reason we use mathematics in computer graphics is:
 - a) To make the code run faster
 - b) To represent and manipulate visual data accurately
 - c) To reduce memory usage
 - d) To simplify GUI programming
- 2- The geometry stage of the graphics pipeline involves:
 - a) Shading and lighting calculations
 - b) Rasterizing triangles into pixels
 - c) Transforming 3D models using matrices
 - d) Managing sound and user input
- 3- In ray tracing, most computation time is spent on:
 - a) Color correction
 - b) Ray-object intersection tests
 - c) Shadow mapping
 - d) Texture compression
- 4- Aliasing in computer graphics refers to:
 - a) Duplicate file names
 - b) Noise in sound rendering
 - c) Stair-step artifacts due to pixelation
 - d) Incorrect lighting equations
- 5- Babylon.js is best described as:
 - a) A low-level GPU driver
 - b) A 3D graphics engine for JavaScript
 - c) A game AI framework
 - d) A photo editing tool
- 6- The purpose of anti-aliasing is to:
 - a) Increase brightness
 - b) Reduce jagged edges
 - c) Speed up rendering
 - d) Reduce file size
- 7- Which of the following is NOT a typical stage in the real-time graphics pipeline?
 - a) Fragment processing

- b) Geometry transformation
- c) Audio synthesis
- d) Rasterization

8- In Data-Oriented Programming (DOP), data is organized primarily by:

- a) Object hierarchy
- b) Access patterns and memory layout
- c) Class inheritance
- d) Encapsulation principles

9- A quaternion can represent:

- a) 2D scaling
- b) 3D rotation
- c) Texture mapping
- d) Lighting intensity

10- The Alpha channel in an image describe ...

- a) Depth
- b) Transparency
- c) Brightness
- d) Color saturation

EPNU-23411015
2026/03/13 13:00

Chapter 2

Graphics Techniques

- 1. Introduction of Rasterization**
- 2. Line Rasterization**
 - 2.1 Digital Differential Analyzer**
 - 2.2 Bresenham's Algorithm**
- 3. Circle Drawing Algorithm**
 - 3.1 Bresenham's Circle**
 - 3.2 Mid-Point Algorithm**
- 4. Polygon Filling Techniques**
 - 4.1 Scan Line Algorithm**
 - 4.1 Boundary Fill Algorithm**
 - 4.2 Flood Fill Algorithm**
- 5. Clipping Methods**
 - 5.1 Cohen Sutherland Method**
 - 5.2 Cyrus – Beck Algorithm**
 - 5.3 Polygon Clipping (Sutherland Hodgeman)**
 - 5.4 Text Clipping**
 - 5.5 Window port to Viewport**

OBJECTIVE

The unit describes the Line Generation, Circle generation Algorithms and Clipping ;

- To Apply Line Generation Algorithm to practical problems;
- Describe the different types of Line Generation Algorithm;
- Describe Circle Generation Algorithm;
- apply Circle Generation algorithm to practical problems;
- describe the different types of Circle Generation Algorithms;
- describe Polygon fill algorithm, and
- describe Scan Line Polygon fill algorithm.

EPNU-23411015
2026/03/13 13:00

1. Introduction of Rasterization

Two-dimensional display devices consist of a discrete grid of pixels, each of which can be independently assigned a color value. **Rasterization (Scan conversion)** is the process of converting two-dimensional primitives into a discrete pixel representation. In other words, the pixels that best describe the primitives must be determined

Scan conversion is the process of converting basic, low level objects into their corresponding pixel map representations. This is often an approximation to the object, since the frame buffer is a discrete grid. Each pixel on the display surface has a finite size depending on the screen resolution and hence, a pixel cannot represent a single mathematical point. However, we consider each pixel as a unit square area identified by the coordinate of its lower left corner, the origin of the reference coordinate system being located at the lower left corner of the display surface. Thus, each pixel is accessed by a non-negative integer coordinate pair (x, y) . The x values start at the origin and increase from left to right along a scan line and the y values.

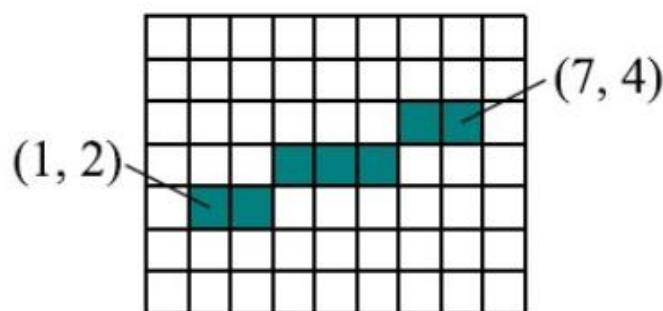


Figure 2.1 Pixel representation of point primitive

Primitive operations involved in C or OpenGL are :

- `setpixel(x, y, color)` : Sets the pixel at position (x, y) to the given color.
- `getpixel(x, y)` : Gets the color at the pixel at position (x, y)

2. Line Rasterization

A line connects two points. Line drawing is accomplished by calculating the intermediate point coordinates along the line path between two given end points. Since, screen pixels are referred with integer values, plotted positions may only approximate the calculated coordinates – i.e., pixels which are intensified are those which lie very close to the line path if not exactly on the line path which is in the case of perfectly horizontal, vertical or 45° lines only. **Standard algorithms** are available to determine which pixels provide the best approximation to the desired line. The explicit equation for a line is $y = mx + b$.

The Basic Line Drawing Algorithm involves the following steps

Step 1: Set the color of pixels to approximate the appearance of a line from (x_0, y_0) to (x_1, y_1) .

Step 2 : Subtract y_0 from y_1 to solve for $m = (y_1 - y_0) / (x_1 - x_0)$ and $b = y_0 - m x_0$.

Step 3 : Substituting in the value for b , this equation can be written as $y = m(x - x_0) + y_0$.

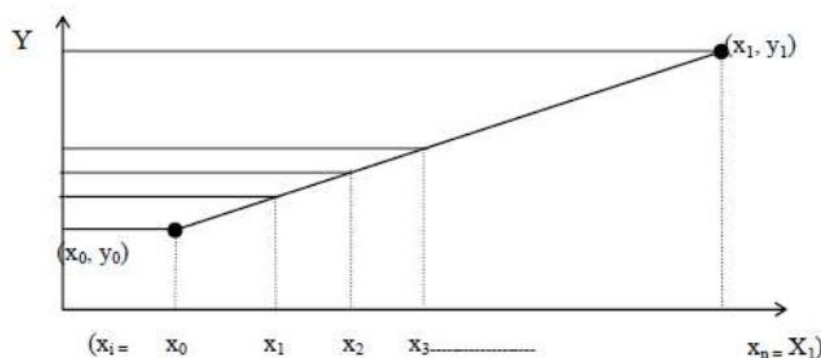


Figure 2 Slope m of the Straight line < 1

2.1 Digital Differential Analyzer (DDA)

Digital Differential Analyzer (DDA) algorithm is the simple line generation algorithm which is explained step by step here.

Step 1 – Get the input of two end points (x_0, y_0) and (x_1, y_1) .

Step 2 – Calculate the difference between two end points.

```
dx =  $x_1 - x_0$ 
```

```
dy =  $y_1 - y_0$ 
```

Step 3 – Based on the calculated difference in step-2, you need to identify the number of steps to put pixel. If $dx > dy$, then you need more steps in x coordinate; otherwise in y coordinate.

```
if (absolute(dx) > absolute(dy))
```

```
Steps = absolute(dx);
```

```
else
```

```
Steps = absolute(dy);
```

Step 4 – Calculate the increment in x coordinate and y coordinate.

```
Xincrement = dx / (float) steps;
```

```
Yincrement = dy / (float) steps;
```

Step 5 – Put the pixel by successfully incrementing x and y coordinates accordingly and complete the drawing of the line.

```
for(int v=0; v < Steps; v++)
```

```
{
```

```
x = x + Xincrement;
```

```
y = y + Yincrement;
```

```
putpixel(Round(x), Round(y));
```

```
}
```

Example 1: Draw line segment from point (2, 4) to (9, 9) using DDA algorithm.

Solution: We know general equation of line is given by (x_0, y_0) and (x_1, y_1) .

$$y = mx + c \text{ where } m = (y_1 - y_0) / (x_1 - x_0)$$

$$\text{given } (x_0, y_0) \rightarrow (2, 4) ; (x_1, y_1) \rightarrow (9, 9)$$

$$\Rightarrow m = (y_1 - y_0) / (x_1 - x_0) = 9 - 4 / 9 - 2$$

$$= 5/7 \text{ i.e., } 0 < m < 1$$

Thus updated pixel values are :- $x_{i+1} = x_i + 1$ $y_{i+1} = y_i + m$

$$\text{given } (x_0, y_0) = (2, 4)$$

$$1) x_1 = x_0 + 1 = 3 \text{ and } y_1 = y_0 + m = 4 + 5/7 = 4.71$$

So, put pixel $(x_1, \text{round}(y_1), \text{colour})$

i.e., put on (3, 5)

$$2) x_2 = x_1 + 1 = 3 + 1 = 4$$

$$y_2 = y_1 + m = (33/7) + 5/7 = 38/7 = 5.428$$

i.e. put on (4, 5)

Similarly go on till (9, 9) is reached.

Example 2:

Scan convert a line having end points (3,2) & (4,7) using DDA.

Solution:

$$dx = x_2 - x_1 = 4 - 3 = 1$$

$$dy = y_2 - y_1 = 7 - 2 = 5$$

As, $dx < dy$ then

$$\text{steps} = \text{absolute}(y_2 - y_1) = 5$$

$$X_{\text{increment}} = dx / (\text{float}) \text{ steps} = 1/5 = 0.2$$

$$Y_{\text{increment}} = dy / (\text{float}) \text{ steps} = 5/5 = 1$$

X	Y	X plotted	Y plotted
3	2	3	2
3.2	3	3	3
3.4	4	3	4
3.6	5	4	5
3.8	6	4	6
4	7	4	7

DDA algorithm

```

line (int x1, int y1, int x2, int y2)
{
    float x,y;
    int dx = x2-x1, dy = y2-y1;
    int n = max(abs(dx),abs(dy));
    float dt = n, dxdt = dx/dt, dydt = dy/dt;
    x = x1;  y = y1;
    while( n-- )          // n - range of t.
    {   point(round(x),round(y));
        x += dxdt;  y += dydt; } }

```

Advantages of DDA

- It calculates the pixel positions faster than the calculations performed by using the equation $y=mx + b$.
- Multiplication is eliminated as the x and y increments are used to determine the position of the next pixel on a line

Disadvantages of DDA

- The rounding and floating point operations are time consuming.
- The round-off error which results in each successive addition leads to the drift in pixel position, already calculated

```
#include <GL/glut.h>
#include <iostream>
#include <cmath> // for abs()

using namespace std;

void putpixel(int x, int y) {
    glBegin(GL_POINTS); // Use OpenGL to draw a point
    glVertex2i(x, y); // Specify pixel position
    glEnd();
    glFlush(); // Render the point
}

void dda(int x1, int y1, int x2, int y2) {
    float dx, dy, steps;
    float x = x1, y = y1; // Initialize x and y to the starting point
    dx = x2 - x1;
    dy = y2 - y1;

    // Find the number of steps to move
    if (abs(dx) > abs(dy))
        steps = abs(dx);
    else
        steps = abs(dy);

    // Calculate the increment for each step
    float Xincrement = dx / steps;
    float Yincrement = dy / steps;

    // Loop through all steps to plot points
    for (int i = 0; i < steps; i++) {
        x += Xincrement;
        y += Yincrement;
        putpixel(round(x), round(y)); // Plot the point
    }
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    dda(50, 100, 300, 300); // Call the DDA function to draw a line
}

void init() {
    glClearColor(1.0, 1.0, 1.0, 1.0); // White background
    glColor3f(0.0, 0.0, 0.0); // Black color for drawing
    glPointSize(2.0); // Set point size
    glMatrixMode(GL_PROJECTION);
    gluOrtho2D(0, 500, 0, 500); // Set coordinate system
```

EPNU - 23411015
2026/03/13 13:00

```

}

int main(int argc, char **argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(500, 500);
    glutCreateWindow("DDA Line Algorithm");
    init();
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}

```

2.2 Bresenham's Line Generation

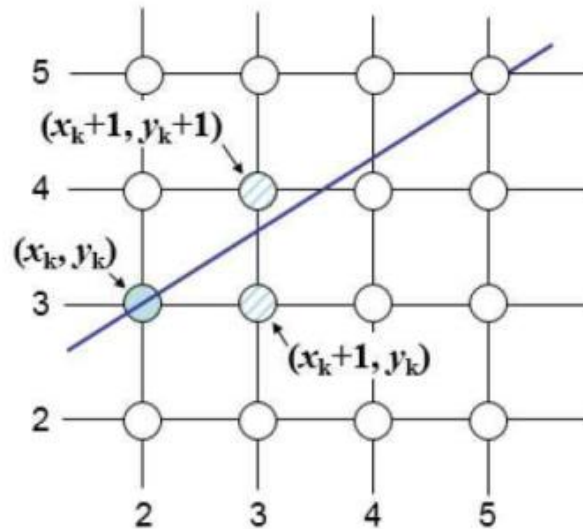
The Bresenham algorithm is another incremental scan conversion algorithm. It is an accurate and efficient raster line generation algorithm. This algorithm scan converts lines using only incremental integer calculations which is a big advantage of this algorithm and these calculations can also be adopted to display circles and other curves moving across the x axis in unit intervals and at each step choose between two different y coordinates. For example, as shown in the following illustration, from position (2, 3) you need to choose between (3, 3) and (3, 4). You would like the point that is closer to the original line.

At sample position X_{k+1}, Y_{k+1} , the vertical separations from the mathematical line are made.

$$d_{\text{new}} = F(x_p + 2, x_p + 1/2) = a(x_p + 2) + b(x_p + 1/2) + c$$

$$\text{but } d_{\text{old}} = a(x_p + 1) + b(y_p + 1/2) + c$$

Now assuming we have to determine that the pixel at (x_k, y_k) is to be displayed, we next need to decide which pixel to plot in column x_{k+1} . Our choices are the pixels at position (x_{k+1}, y_k) and (x_{k+1}, y_{k+1}) .



At sampling position x_{k+1} , we label vertical pixel separations from the mathematical line path as d_1 and d_2 . The y coordinate on the mathematical line at pixel column position x_{k+1} is calculated as : $y = m(x_k + 1) + b$

Then $d_1 = y - y_k = m(x_k + 1) + b - y_k$

and $d_2 = (y_k + 1) - y = y_k + 1 - m(x_k + 1) - b$

The difference between these two separations is

$$d_1 - d_2 = 2m(x_k + 1) - 2y_k + 2b - 1$$

A decision Parameter P_k for the kth step in the line algorithm can be obtained by rearranging and by substituting $m = \Delta y / \Delta x$. where Δy & Δx are the vertical & horizontal separation of the endpoint positions & defining.

$$P_k = \Delta x (d_1 - d_2) = 2\Delta y \cdot x_k - 2\Delta x y_k + c \quad \text{--- (A)}$$

The sign of P_k is same as the sign of $d_1 - d_2$

This recursive calculation of decision Parameter is performed each integer x position, starting at left coordinate endpoint of the line. The first parameter P_0 is evaluated from equation (A) at starting pixel position (x_0, y_0) and with m evaluated as $\Delta y / \Delta x$.

$$P_0 = 2\Delta y - \Delta x \quad \text{--- (B)}$$

Since $\Delta x > 0$ for our example Parameter C is constant & has the value $2\Delta y + \Delta x$ (2b

-1), which is independent of pixel position. If the pixel position at y_k is closer to line path than the pixel at y_{k+1} (that is $d_1 < d_2$), then decision Parameter P_k is Negative. In that case we plot the lower pixel otherwise we plot the upper pixel.

i.e. $P_{k+1} = P_k + 2 * \Delta y$;

else when Decision Parameter $P_k > 0$

$y_{k+1} = y_{k+1}$; $x_{k+1} = x_{k+1}$

and $P_{k+1} = P_k + 2 * (\Delta y - \Delta x)$;

Coordinate changes along the line owner in unit steps in either the x or directions. Therefore we can obtain the values of successive decision Parameter using incremental integer calculations.

Example 3: Digitize the line with end points (20, 10) & (30, 18) using Bresenham's Line Drawing Algorithm

Hint Solution :

$\Delta x = 10$, $\Delta y = 8$

Initial decision parameter has the value

$$P_0 = 2\Delta y - \Delta x = 2 \times 8 - 10 = 6$$

Since $P_0 > 0$, so next point is (x_{k+1}, y_{k+1}) (21, 11)

Now $k = 0$, $P_{k+1} = P_k + 2\Delta y - 2\Delta x$

$$P_1 = P_0 + 2\Delta y - 2\Delta x$$

$$= 6 + (-4)$$

$$= 2$$

Since $P_1 > 0$, Thus , Next point is (22, 12)

Now $k = 1$, $P_{k+1} = P_k + 2\Delta y - 2\Delta x$

$$P_2 = 2 + (-4)$$

$$= -2$$

Since $P_2 < 0$, Thus, Next point is (23, 12)

Now $k = 2$ $P_{k+1} = P_k + 2\Delta y$

$$P_2 = -2 + 16$$

$$= 14$$

Since $P_3 > 0$, Thus, Next point is (24, 13)

$$\text{Now } k = 3 \quad P_{k+1} = P_k + 2\Delta y - 2\Delta x$$

$$P_4 = 14 - 4$$

$$= 10$$

Since $P_4 > 0$, So, Next point is (25, 14)

$$\text{Now } k = 4 \quad P_{k+1} = P_k + 2\Delta y - 2\Delta x$$

$$P_5 = 10 - 4$$

$$= 6$$

Since $P_5 > 0$, ••Next point is (26, 15)

$$\text{Now } k = 5 \quad P_{k+1} = P_k + 2\Delta y - 2\Delta x$$

$$P_6 = 6 - 4$$

$$= 2$$

Since $P_6 > 0$, So,Next point is (27, 16)

$$\text{Now } k = 6 \quad P_{k+1} = P_k + 2\Delta y - 2\Delta x$$

$$P_7 = 2 + (-4)$$

$$= -2$$

Since $P_7 < 0$, So, Next point is (28, 16)

$$\text{Now } k = 7 \quad P_{k+1} = P_k + 2\Delta y$$

$$P_8 = -2 + 16$$

$$= 14$$

Since $P_8 > 0$, So, Next point is (29, 17)

$$\text{Now } k = 8 \quad P_{k+1} = P_k + 2\Delta y - 2\Delta x$$

$$P_9 = 14 - 4$$

$$= 10$$

Since $P_9 > 0$, So, Next point is (30, 18)

i.e. To Summarize the Raster Points

K	P _k	x _{k+1}	y _{k+1}
0	6	21	11
1	2	22	12
2	-2	23	12
3	14	24	13
4	10	25	14
5	6	26	15
6	2	27	16
7	-2	28	16
8	14	29	17
9	10	30	18

Example:

•Let the given end points for the line be (30,20) and (40, 28) using Bresenham's Algorithm.

solution:

$$M = dy/dx = (y_2 - y_1)/(x_2 - x_1) = (28 - 20)/(40 - 30) = 0.8$$

dy= 8 and dx = 10 Major=x , Minor=y

Bx > Ax increment one to x

By > Ay increment one to y

The initial decision parameter P₀ is

$$P_0 = 2dy - dx = 2(8) - 10 = 16 - 10 = 6$$

•The constants 2dy and 2dy-2dx are

$$2dy = 2(8) = 16 \quad 2dy - 2dx = 2(8) - 2(10) = 16 - 20 = -4$$

$$2dy = 16 \quad 2dy - 2dx = -4$$

The starting point (x₀, y₀)=(30,20) and the successive pixel positions are given in the following table:

i	P _i	(X _{i+1} , Y _{i+1})
0	6	(31,21)
1	2	(32,22)
2	-2	(33,22)
3	14	(34,23)
4	10	(35,24)
5	6	(36,25)
6	2	(37,26)
7	-2	(38,26)
8	14	(39,27)
9	10	(40,28)

End points (30,20) (40,28)

$dx = x_2 - x_1 = 40 - 30 = 10$ $dy = y_2 - y_1 = 28 - 20 = 8$

$P_0 = 2dy - dx = 2 \times 8 - 10 = 6 > 0$

$P_{i+1} = p_i + 2dy - 2dx = 6 + 16 - 20 = 2 > 0$

$P_{i+1} = 2 + 16 - 20 = -2 < 0$

$P_{i+1} = p_i + 2dy = -2 + 16 = 14 > 0$

$P_{i+1} = p_i + 2dy - 2dx = 14 + 16 - 20 = 10 > 0$

$P_{i+1} = 10 + 16 - 20 = 6 > 0$

$P_{i+1} = 6 + 16 - 20 = 2 > 0$

$P_{i+1} = 2 + 16 - 20 = -2 < 0$

$P_{i+1} = p_i + 2dy = -2 + 16 = 14 > 0$

$P_{i+1} = p_i + 2dy - 2dx = 14 + 16 - 20 = 10 > 0$

pt(31,21)

(32,22)

(33,22)

(34,23)

(35,24)

(36,25)

(37,26)

(38,26)

(39,27)

(40,28)

3. Circle Drawing Algorithm

A circle is a set of points that are at a given distance r from the center position (x_c, y_c) . This distance relationship is given as : $(x - x_c)^2 + (y - y_c)^2 - r^2 = 0$. We cannot display a continuous arc on the raster display. Instead, we have to choose the nearest pixel position to complete the arc. This equation is used to calculate the position of points along the circle path by moving in the x direction from $(x_c - r)$ to $(x_c + r)$ and we have put the pixel at (X, Y) location and now need to decide where to put the next pixel – at N $(X+1, Y)$ or at S $(X+1, Y-1)$.

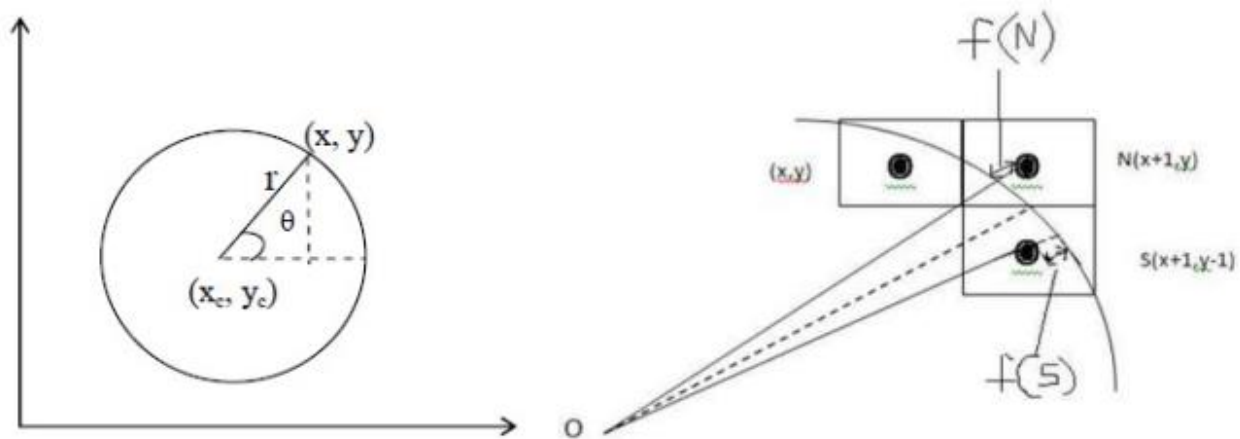


Figure Circle Generation Algorithm with polar coordinates

By calculating the polar coordinates r and θ where

$$x = x_c + r \cos \theta$$

$$y = y_c + r \sin \theta$$

Thus, an efficient approach based on incremental calculations of decision parameter is employed for circle generation. There are two popular algorithms for generating a circle – Bresenham's Algorithm and Midpoint Circle Algorithm. These algorithms are based on the idea of determining the subsequent points required to draw the circle.

For a given radius and center position (x_c, y_c) we first setup our algorithm to calculate pixel position around the path of circle centered at coordinate origin $(0, 0)$ *i.e.*, we translate $(x_c, y_c) \rightarrow (0, 0)$ and after the generation we do inverse translation $(0, 0) \rightarrow (x_c, y_c)$

hence each calculated position (x, y) of circumference is moved to its proper screen position by adding x_c to x and y_c to y .

3.1 Bresenham's Circle Generation

Here, the decision parameter d is used to rasterize the next point on the circle.

- If $d \leq 0$, then $N(X+1, Y)$ is to be chosen as next pixel.
- If $d > 0$, then $S(X+1, Y-1)$ is to be chosen as the next pixel.

Step 1 – Get the coordinates of the center of the circle and radius, and store them in x , y , and R respectively. Set $P=0$ and $Q=R$.

Step 2 – Set decision parameter $D = 3 - 2R$.

Step 3 – Repeat through step-8 while $X < Y$.

Step 4 – Call Draw Circle (X, Y, P, Q).

Step 5 – Increment the value of P .

Step 6 – If $D < 0$ then $D = D + 4x + 6$.

Step 7 – Else Set $Y = Y + 1$, $D = D + 4(X-Y) + 10$.

Step 8 – Call Draw Circle (X, Y, P, Q).

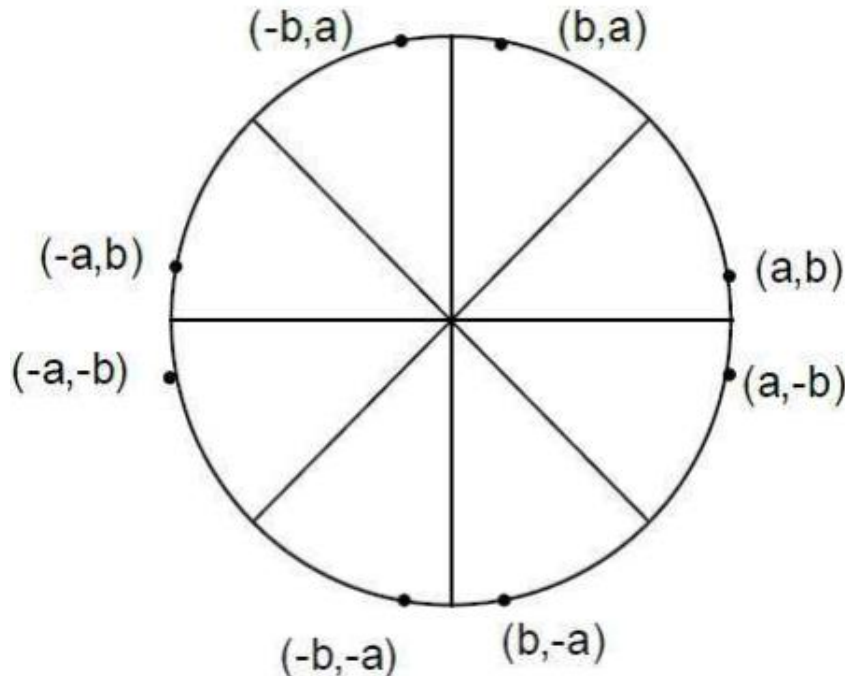


Figure Quadrant end-points of drawing the circle

```
#include<GL/glut.h>
```

```
#include<math.h>
```

```
const float Deg2rad =3.14159/180;
void circle(){
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0, 0.0, 0.0);      // Black color for drawing
    float raduis=200;
    glBegin(GL_LINE_LOOP);
    for(int i=0;i<=360;i++)
    {
        float degInrad =i*Deg2rad;
        glVertex2f (300+cos(degInrad)*raduis,300+sin(degInrad)*raduis);
    }
    glEnd();
    glFlush();
}

void init()
{
    glClearColor(1.0, 1.0, 1.0, 1.0); // White background
    glPointSize(2.0);                  // Set point size
    glMatrixMode(GL_PROJECTION);
    gluOrtho2D(0, 600, 0, 600);       // Set coordinate system
}

int main(int argc, char **argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(500, 500);
```

```

glutCreateWindow("Draw circle");
init();
glutDisplayFunc(circle);
glutMainLoop();
return 0;
}

```

3.1 Mid-Point Circle Generation

1. Decision Parameter d_i :

- The function $f(x, y) = x^2 + y^2 - r^2$ determines whether a point is inside, outside, or on the circle.
- The midpoint approach evaluates the function at the midpoint between two possible pixel choices.

2. Choosing the Next Pixel:

- If $d_i > 0$, select point S: $(x_i - 1, y_i + 1)$, moving diagonally.
- If $d_i \leq 0$, select point T: $(x_i, y_i + 1)$, moving directly up.

3. Decision Parameter Update Rules:

- If S is chosen:

$$d_{i+1} = d_i + 2(y_i + 1 - x_i + 1) + 1$$

- If T is chosen:

$$d_{i+1} = d_i + 2y_i + 1$$

4. Initial Decision Parameter:

- $d_0 = 5/4 - r$ (or $1 - r$ if r is an integer).

4. Polygon Filling Techniques

Polygon can be defined as an ordered list of vertices that is formed by line segments that are placed end to end, creating a continuous closed path. Polygons can be divided into three basic types: convex, concave, and complex.

Types of Polygons

Regular – all angles are equal and all sides are the same length. Regular polygons are both equiangular and equilateral.

Equiangular – all angles are equal.

Equilateral – all sides are the same length.

Convex – a straight line drawn through a convex polygon **crosses at most two sides**. Every interior angle is less than 180° . **Convex polygons** are the simplest type of polygon to fill.

Concave – you can draw at least one straight line through a concave polygon that **crosses more than two sides**. At least one interior angle is more than 180° .

Polygon Formulas

(N = # of sides and S = length from center to a corner)

Area of a regular polygon = $(1/2) N \sin(360^\circ/N) S^2$

Sum of the interior angles of a polygon = $(N - 2) \times 180^\circ$

The **number of diagonals** in a polygon = $1/2 N * (N-3)$

In order to fill a polygon, we do not want to have to determine the type of polygon that we are filling. The easiest way to avoid this situation is to use an algorithm that works for all three types of polygons. Since both convex and concave polygons are subsets of the complex type, using an algorithm that will work for complex polygon filling should be sufficient for all three types.

For filling polygons with particular colors, you need to determine the pixels falling on the border of the polygon and those which fall inside the polygon. There are two basic approaches to filling on raster systems. One way is Line Filling approach which is adopted later in area filling method by drawing straight lines between the edges of

polygon called scan-line polygon filling. Second way is to start from an interior point and paint outward from this point till we reach the boundary called boundary-fill. A slight variation of this technique is used to fill an area specified by cluster (having no specific boundary). The technique is called flood-fill and having almost same strategy that is to start from an interior point and start painting outward from this point till the end of cluster.

Now having an idea we will try to see each of these one by one, starting from scan-line polygon filling.

5. Scan Line Algorithm

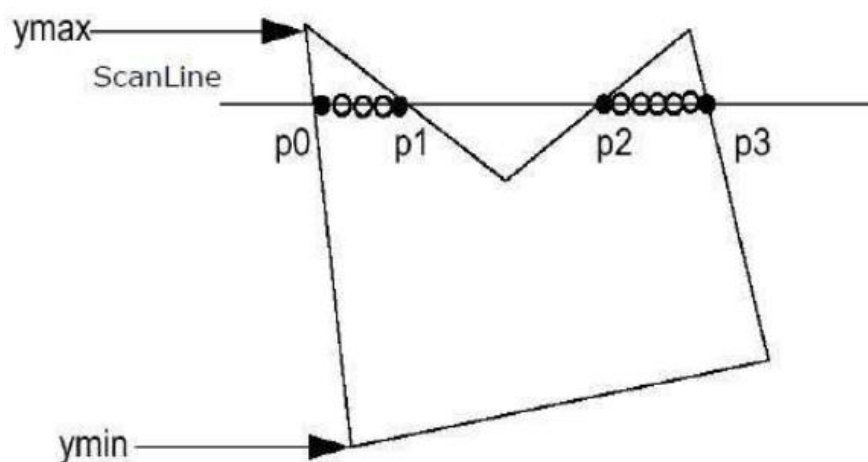
What is a scan-line?

A scan-line is a line of constant y value, i.e., $y=c$, where c lies within our drawing region, e.g., the window on our computer screen.

This algorithm works by intersecting scanline with polygon edges and fills the polygon between pairs of intersections.

The following steps depict how this algorithm works.

Step 1 – Find out the $Y_{\min}$ and $Y_{\max}$ from the given polygon.



Step 2 – ScanLine intersects with each edge of the polygon from $Y_{\min}$ to $Y_{\max}$.

Name each intersection point of the polygon. As per the figure shown above, they

are named as p_0, p_1, p_2, p_3 .

Step 3 – Sort the intersection point in the increasing order of X coordinate i.e. (p_0, p_1) , (p_1, p_2) , and (p_2, p_3) .

Step 4 – Fill all those pair of coordinates that are inside polygons and ignore the alternate pairs.

6. Boundary Fill Algorithm

Another important class of area-filling algorithms starts at a point known to be inside a figure and starts filling in the figure outward from the point. Using these algorithms a graphic artist may sketch the outline of a figure and then select a color or pattern with which to fill it. The actual filling process begins when a point inside the figure is selected. These routines are like the paint scan function seen in common interactive paint packages.

The boundary-fill method requires the coordinates of a starting point, a fill color, and a boundary color as arguments.

The Boundary fill algorithm performs the following steps:

Check the pixel for boundary color

Check the pixel for fill color

Set the pixel in fill color

Run the process for neighbors

BOUNDARY FILL PSEUDOCODE:

```
boundaryFill (x, y, fillColor, boundaryColor)
if ((x < 0) || (x >= width)) return
if ((y < 0) || (y >= height)) return
current = GetPixel(x, y)
if ((current != boundaryColor) && (current != fillColor))
setPixel(fillColor, x, y);
```

```
boundaryFill (x+1, y, fillColor, boundaryColor)
boundaryFill (x, y+1, fillColor, boundaryColor)
boundaryFill (x-1, y, fillColor, boundaryColor)
boundaryFill (x, y-1, fillColor, boundaryColor)
```

Note that this is a **recursive routine**. Each invocation of `boundaryFill ()` may call itself four more times.

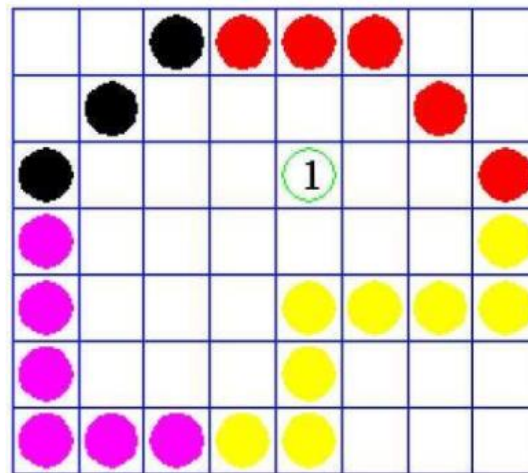
The logic of this routine is very simple. If we are not either on a boundary or already filled we first fill our point, and then tell our neighbors to fill themselves.

The boundary fill algorithm works as its name. This algorithm picks a point inside an object and starts to fill until it hits the boundary of the object. The color of the boundary and the color that we fill should be different for this algorithm to work.

7. Flood Fill Algorithm

Sometimes we come across an object where we want to fill the area and its boundary with different colors. We can paint such objects with a specified interior color instead of searching for particular boundary color as in boundary filling algorithm. Instead of relying on the boundary of the object, it relies on the fill color. In other words, it replaces the interior color of the object with the fill color. When no more pixels of the original interior color exist, the algorithm is completed.

Once again, this algorithm relies on the Four-connect or Eight-connect method of filling in the pixels. But instead of looking for the boundary color, it is looking for all adjacent pixels that are a part of the interior.



In this algorithm, we assume that color of the boundary is same for the entire object. The boundary fill algorithm can be implemented by 4-connected pixels or 8-connected pixels.

4-Connected Polygon

In this technique 4-connected pixels are used as shown in the figure. We are putting the pixels above, below, to the right, and to the left side of the current pixels and this process will continue until we find a boundary with different color.

4- Connected Boundary Fill Algorithm

Step 1 – Initialize the value of seed point (seedx, seedy), fcolor and dcol.

Step 2 – Define the boundary values of the polygon.

Step 3 – Check if the current seed point is of default color, then repeat the steps 4 and 5 till the boundary pixels reached.

If `getpixel(x, y) = dcol` then repeat step 4 and 5

Step 4 – Change the default color with the fill color at the seed point.

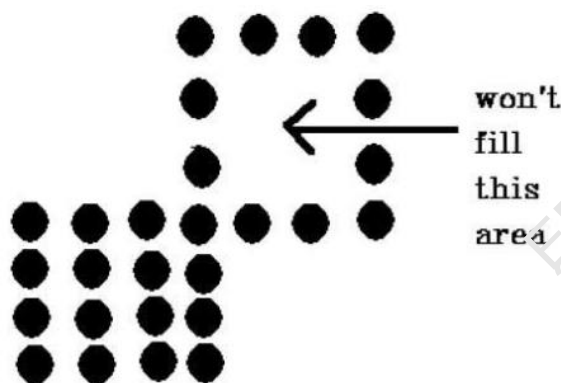
`setPixel(seedx, seedy, fcol)`

Step 5 – Recursively follow the procedure with four neighborhood points.

```
FloodFill (seedx - 1, seedy, fcol, dcol)
FloodFill (seedx + 1, seedy, fcol, dcol)
FloodFill (seedx, seedy - 1, fcol, dcol)
FloodFill (seedx - 1, seedy + 1, fcol, dcol)
```

Step 6 – Exit

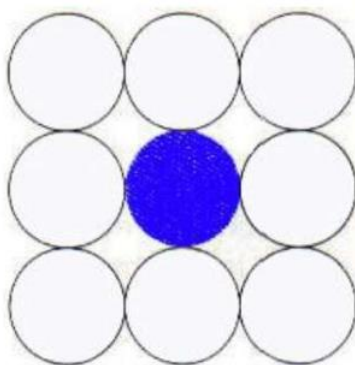
There is a problem with this technique. Consider the case as shown below where we tried to fill the entire region. Here, the image is filled only partially. In such cases, 4-connected pixels technique cannot be used.



8-Connected Polygon

In this technique 8-connected pixels are used as shown in the figure. We are putting pixels above, below, right and left side of the current pixels as we were doing in 4-connected technique.

In addition to this, we are also putting pixels in diagonals so that entire area of the current pixel is covered. This process will continue until we find a boundary with different color.



8– Connected Boundary Fill Algorithm

Step 1 – Initialize the value of seed point (seedx, seedy), fcolor and dcol.

Step 2 – Define the boundary values of the polygon.

Step 3 – Check if the current seed point is of default color then repeat the steps 4 and 5 till the boundary pixels reached

If $\text{getPixel}(x,y) = \text{dcol}$ then repeat step 4 and 5

Step 4 – Change the default color with the fill color at the seed point.

$\text{setPixel}(\text{seedx}, \text{seedy}, \text{fcol})$

Step 5 – Recursively follow the procedure with four neighbourhood points

$\text{FloodFill}(\text{seedx} - 1, \text{seedy}, \text{fcol}, \text{dcol})$

$\text{FloodFill}(\text{seedx} + 1, \text{seedy}, \text{fcol}, \text{dcol})$

$\text{FloodFill}(\text{seedx}, \text{seedy} - 1, \text{fcol}, \text{dcol})$

$\text{FloodFill}(\text{seedx}, \text{seedy} + 1, \text{fcol}, \text{dcol})$

$\text{FloodFill}(\text{seedx} - 1, \text{seedy} + 1, \text{fcol}, \text{dcol})$

$\text{FloodFill}(\text{seedx} + 1, \text{seedy} + 1, \text{fcol}, \text{dcol})$

$\text{FloodFill}(\text{seedx} + 1, \text{seedy} - 1, \text{fcol}, \text{dcol})$

$\text{FloodFill}(\text{seedx} - 1, \text{seedy} - 1, \text{fcol}, \text{dcol})$

Step 6 – Exit

The 4-connected pixel technique failed to fill the area as marked in the following figure which won't happen with the 8-connected technique.

Fill Boundary_1

```
#include <GL/glut.h>
#include <cmath>
const int width = 600, height = 600; // Screen dimensions

// Function to set a pixel at (x, y) with a specific color
void setPixel(int x, int y, float color[3]) {
    glColor3fv(color);
    glBegin(GL_POINTS);
    glVertex2i(x, y);
    glEnd();
    glFlush();
}

// Function to get the color of a pixel at (x, y) (Placeholder)
void getPixelColor(int x, int y, float color[3]) {
    glReadPixels(x, y, 1, 1, GL_RGB, GL_FLOAT, color);
}

// Recursive boundary fill function
void boundaryFill(int x, int y, float fillColor[3], float boundaryColor[3]) {
    if (x < 0 || x >= width || y < 0 || y >= height) return;

    float currentColor[3];
    getPixelColor(x, y, currentColor);

    // Check if the current pixel is neither the boundary color nor the fill color
    if ((currentColor[0] != boundaryColor[0] || currentColor[1] != boundaryColor[1] || currentColor[2] !=
boundaryColor[2]) &&
        (currentColor[0] != fillColor[0] || currentColor[1] != fillColor[1] || currentColor[2] != fillColor[2])) {
        setPixel(x, y, fillColor);

        // Recursively fill neighboring pixels
        boundaryFill(x + 1, y, fillColor, boundaryColor);
        boundaryFill(x - 1, y, fillColor, boundaryColor);
        boundaryFill(x, y + 1, fillColor, boundaryColor);
        boundaryFill(x, y - 1, fillColor, boundaryColor);
    }
}
```

```
// Function to draw a polygon
void drawPolygon() {
    glClear(GL_COLOR_BUFFER_BIT);

    float boundaryColor[3] = {0.0f, 0.0f, 0.0f}; // Black boundary color
    float fillColor[3] = {1.0f, 0.0f, 0.0f};    // Red fill color

    // Draw polygon
    glColor3fv(boundaryColor);
    glBegin(GL_LINE_LOOP);
    glVertex2i(200, 200);
    glVertex2i(400, 200);
    glVertex2i(450, 350);
    glVertex2i(300, 450);
    glVertex2i(150, 350);
    glEnd();
    glFlush();

    // Start boundary fill from a point inside the polygon
    boundaryFill(300, 300, fillColor, boundaryColor);
}

void init() {
    glClearColor(1.0, 1.0, 1.0, 1.0); // White background
    glPointSize(2.0);                 // Set point size
    glMatrixMode(GL_PROJECTION);
    gluOrtho2D(0, width, 0, height); // Set coordinate system
}

int main(int argc, char **argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(width, height);
    glutCreateWindow("Boundary Fill - Polygon");
    init();
    glutDisplayFunc(drawPolygon);
    glutMainLoop();
    return 0;
}
```


Fill Boundary_2

```
#include <GL/glut.h>
#include <cmath>

const float Deg2rad = 3.14159 / 180.0;
int width = 600, height = 600; // Define the screen width and height

// Function to set a pixel at (x, y) with a specific color
void setPixel(int x, int y, float color[3]) {
    glColor3fv(color);
    glBegin(GL_POINTS);
    glVertex2i(x, y);
    glEnd();
    glFlush();
}

// Function to get the color of a pixel at (x, y) (Placeholder)
void getPixelColor(int x, int y, float color[3]) {
    glReadPixels(x, y, 1, 1, GL_RGB, GL_FLOAT, color);
}

// Recursive boundary fill function
void boundaryFill(int x, int y, float fillColor[3], float boundaryColor[3]) {
    if (x < 0 || x >= width || y < 0 || y >= height) return;

    float currentColor[3];
    getPixelColor(x, y, currentColor);

    // Check if the current pixel is neither the boundary color nor the fill color
    if ((currentColor[0] != boundaryColor[0] || currentColor[1] != boundaryColor[1] || currentColor[2] !=
boundaryColor[2]) &&
        (currentColor[0] != fillColor[0] || currentColor[1] != fillColor[1] || currentColor[2] != fillColor[2])) {
        setPixel(x, y, fillColor);

        // Recursively fill neighboring pixels
        boundaryFill(x + 1, y, fillColor, boundaryColor);
        boundaryFill(x - 1, y, fillColor, boundaryColor);
        boundaryFill(x, y + 1, fillColor, boundaryColor);
        boundaryFill(x, y - 1, fillColor, boundaryColor);
    }
}

// Function to draw a circle (or any initial shape)
void circle() {
    glClear(GL_COLOR_BUFFER_BIT);
```

```
float boundaryColor[3] = {0.0f, 0.0f, 0.0f}; // Black boundary color
float fillColor[3] = {1.0f, 0.0f, 0.0f};    // Red fill color

// Draw circle
int xc = 300, yc = 300, radius = 100;
glColor3fv(boundaryColor);
glBegin(GL_LINE_LOOP);
for (int i = 0; i < 360; i++) {
    float theta = i * Deg2rad;
    glVertex2i(xc + radius * cos(theta), yc + radius * sin(theta));
}
glEnd();
glFlush();

// Start boundary fill from a point inside the circle
boundaryFill(xc, yc, fillColor, boundaryColor);
}

void init() {
    glClearColor(1.0, 1.0, 1.0, 1.0); // White background
    glPointSize(2.0);                // Set point size
    glMatrixMode(GL_PROJECTION);
    gluOrtho2D(0, width, 0, height); // Set coordinate system
}

int main(int argc, char **argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(width, height);
    glutCreateWindow("Boundary Fill Algorithm");
    init();
    glutDisplayFunc(circle);
    glutMainLoop();
    return 0;
}
```

8. Clipping Methods

Clipping may be described as the procedure that identifies the portions of a picture lie inside the region, and therefore, should be drawn or, outside the specified region, and hence, not to be drawn. The algorithms that perform the job of clipping are called clipping algorithms there are various types, such as:

- **Point Clipping**
- **Line Clipping**
- **Polygon Clipping**
- **Text Clipping**
- **Curve Clipping**

Further, there are a wide variety of algorithms that are designed to perform certain types of clipping operations, some of them which will be discussed in unit.

Line Clipping Algorithms:

- Cohen Sutherland Line Clippings
- Cyrus–Beck Line Clipping Algorithm

Polygon or Area Clipping Algorithm

- Sutherland–Hodgman Algorithm

Before going into the details of point clipping, let us look at some basic terminologies used in the field of clipping, such as, window and viewport. Window may be described as the world coordinate area selected for display.

Viewport may be described as the area on a display device on which the window is mapped.

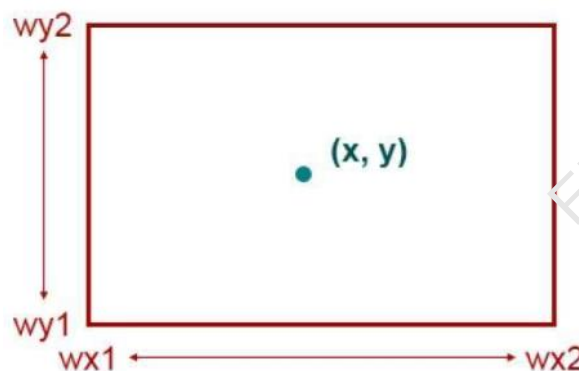
So, it is the window that specifies what is to be shown or displayed whereas viewport specifies where it is to be shown or displayed. Specifying these two coordinates, i.e., window and viewport coordinates and then the transformation from window to viewport coordinates is very essential from the point of view of clipping.

Note: Assumption: That the window and viewport are rectangular. Then only, by specifying the maximum and the minimum coordinates i.e., (X_{wmax}, Y_{wmax}) and (X_{wmin}, Y_{wmin}) we can describe the size of the overall window or viewport.

- Window and viewport are not restricted to only rectangular shapes they could be of any other shape (Convex or Concave or both).

1. Point clipping tells us whether the given point (X, Y) is within the given window or not; and decides whether we will use the minimum and maximum coordinates of the window.

The X-coordinate of the given point is inside the window, if X lies in between $W_{x1} \leq X \leq W_{x2}$. The Y coordinate of the given point is inside the window, if Y lies in between $W_{y1} \leq Y \leq W_{y2}$.

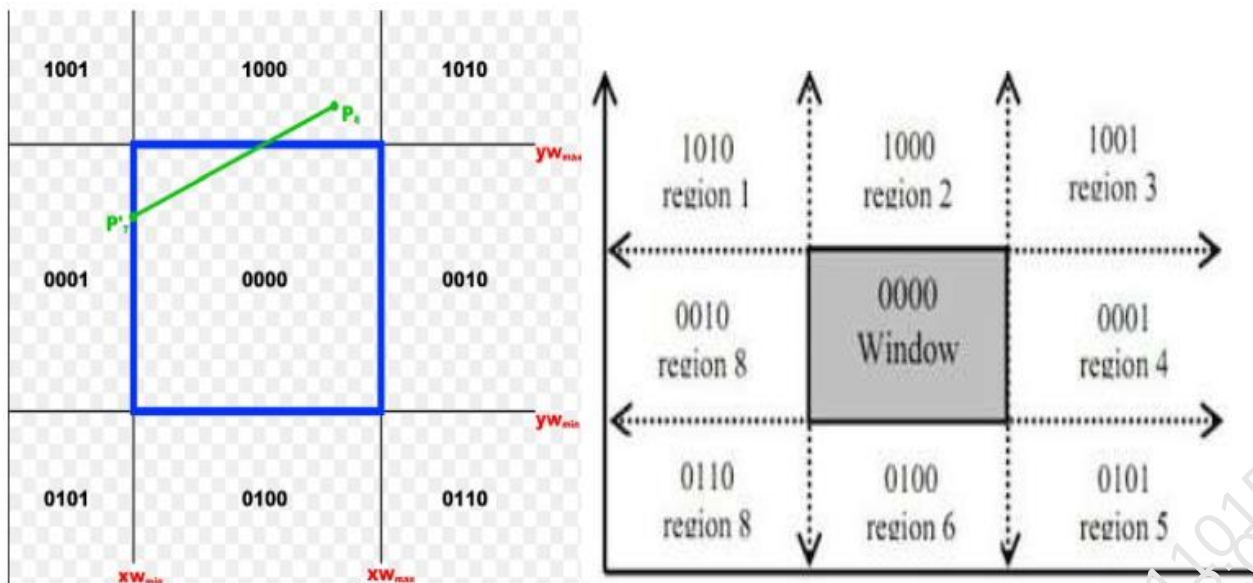


2. line clipping: A variety of line clipping algorithms are available in the world of computer graphics, two of which are :

- 1) Cohen Sutherland algorithm,
- 2) Cyrus–Beck of algorithm

8.1 COHEN SUTHERLAND LINE CLIPPING

The clipping problem is simplified by dividing the area surrounding the window region into four segments Up, Down, Left, Right (U, D,L,R) and assignment of number 1 and 0 to respective segments helps in positioning the region surrounding the window. How this positioning of regions is performed can be well understood by considering the Figure below.



All coding of regions U,D,L,R is done with respect to window region. As window is neither Up nor Down, neither Left nor Right, so, the respective bits UDLR are 0000 of Central region. The positioning code UDLR is 1010, i.e., the region 1 lying on the position which is upper left side of the window. Thus, region 1 has UDLR code 1010 (Up so U=1, not Down so D=0, Left so L=1, not Right so R=0).

The meaning of the UDLR code to specify the location of region with respect to window is:

1st bit $\Rightarrow$ Up(U) ; 2nd bit $\Rightarrow$ Down(D) ; 3rd bit $\Rightarrow$ Left(L) ; 4th bit $\Rightarrow$ Right(R),

Now, to perform Line clipping for various line segment which may reside inside the window region fully or partially, or may not even lie in the window region; we use the tool of logical ANDing between the UDLR codes of the points lying on the line. Logical ANDing ($\wedge$) operation $\Rightarrow 1 \wedge 1 = 1; 1 \wedge 0 = 0;$

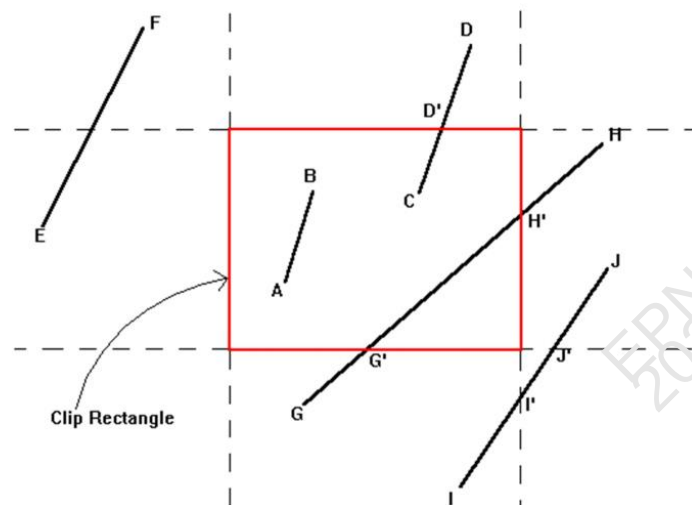
between respective bits implies $0 \wedge 1 = 0; 0 \wedge 0 = 0$

Note:

- ✓ UDLR code of window is 0000 always and w.r.t. this will create bit codes of other regions.

- ✓ **A line segment is visible if both the UDLR codes of the end points of the line segment equal to 0000 i.e. UDLR code of window region. If the resulting code is not 0000 then, that line segment or section of line segment may or may not be visible.**

Now, let us study how this clipping algorithm works. For the sake of simplicity we will tackle all the cases with the help of example lines 11 to 15 shown in Figure.



There are 3 possibilities for the line –

- Line can be completely inside the window (This line should be accepted).
- Line can be completely outside of the window (This line will be completely removed from the region).
- Line can be partially inside the window (We will find intersection point and draw only that portion of line that is inside region).

Algorithm

Step 1 – Assign a region code for each endpoints.

Step 2 – If both endpoints have a region code 0000 then accept this line.

Step 3 – Else, perform the logical AND operation for both region codes.

Step 3.1 – If the result is not 0000, then reject the line.

Step 3.2 – Else you need clipping.

Step 3.2.1 – Choose an endpoint of the line that is outside the window.

Step 3.2.2 – Find the intersection point at the window boundary (base on region code).

Step 3.2.3 – Replace endpoint with the intersection point and update the region code.

Step 3.2.4 – Repeat step 2 until we find a clipped line either trivially accepted or trivially rejected.

Step 4 – Repeat step 1 for other lines.

Example : For the rectangular window boundaries given as $x_L=2$, $y_B=2$, $x_R=8$, $y_T=8$, check the visibility of the following segments using the Cohen–Sutherland algorithm and, if necessary, clip them against the appropriate window boundaries.

- Line AB: A(3,10) B(6,12);
- Line CD: C(4,1), D(10,6)

Solution: –

a) **Step 1.** Set up the end codes of the two lines

$A(3,10) \rightarrow (1000)$; $B(6,12) \rightarrow (1000)$; $\rightarrow$ Line AB is invisible

$C(4,1) \rightarrow (0100)$; $D(10,6) \rightarrow (0010)$; $\rightarrow$ Line CD is indeterminate

Step 2. Clipping of line CD

(a) Endpoint C has a code of (0100). Since bit 2 is not zero, intersection must be found with the boundary $y=y_0=2$. The parametric equation of line CD is

$$X = 4 + (10-4)t = 4 + 6t$$

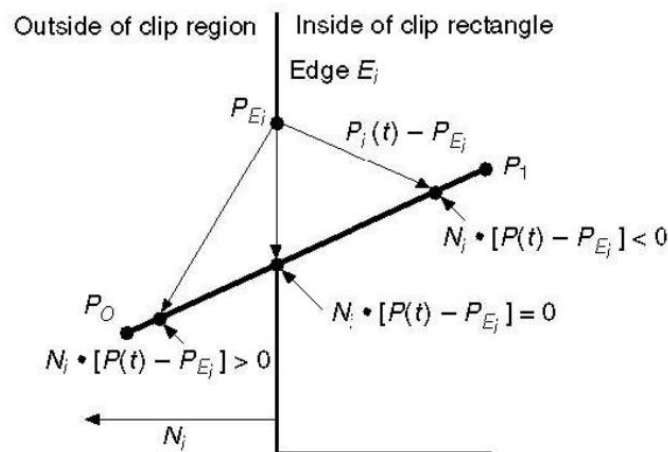
$$Y = 1 + (6-1)t = 1 + 5t$$

Solving $\rightarrow t=0.2$ so intersection point is **I1=(5.2,2)**

- b) Endpoint D has a code of (0010). For bit 3 not equal to zero, the intersection with the boundary $x=x_R=8$ must be found. Substituting $x=8$ to previous equation yields $t=4/6=0.667$ and $y=4.33$. So the intersection point is $I_2(8,4.33)$. Since both I_1 and I_2 lie on the window boundary, their end codes are (0000) and (0000), respectively. The line segment is, therefore, visible between the two intersection points.

8.2 Cyrus–Beck Line Clipping Algorithm

The Cyrus–Beck algorithm is of $O(N)$ complexity, and it is primarily intended for a clipping a line in the parametric form against a convex polygon in 2 dimensions. It was designed to be more efficient than the Sutherland Cohen algorithm which uses repetitive clipping. It was introduced back in 1978 by Cyrus and Beck and it employs parametric line representation and simple dot products.



Parametric equation of line is –

$$P_0P_1: P(t) = P_0 + t(P_1 - P_0)$$

Let N_i be the outward normal edge E_i . Now pick any arbitrary point P_{E_i} on edge E_i then the dot product $N_i \cdot [P(t) - P_{E_i}]$ determines whether the point $P(t)$ is “inside the clip edge” or “outside” the clip edge or “on” the clip edge.

The point $P(t)$ is inside if $N_i.[P(t) - P_{Ei}] < 0$

The point $P(t)$ is outside if $N_i.[P(t) - P_{Ei}] > 0$

The point $P(t)$ is on the edge if $N_i.[P(t) - P_{Ei}] = 0$ (Intersection point) $N_i.[P(t) - P_{Ei}] = 0$

$N_i.[P_0 + t(P_1 - P_0) - P_{Ei}] = 0$ (Replacing $P(t)$ with $P_0 + t(P_1 - P_0)$)

$N_i.[P_0 - P_{Ei}] + N_i.t[P_1 - P_0] = 0$

$N_i.[P_0 - P_{Ei}] + N_i.tD = 0$ (substituting D for $[P_1 - P_0]$)

$N_i.[P_0 - P_{Ei}] = - N_i.tD$

The equation for t becomes,

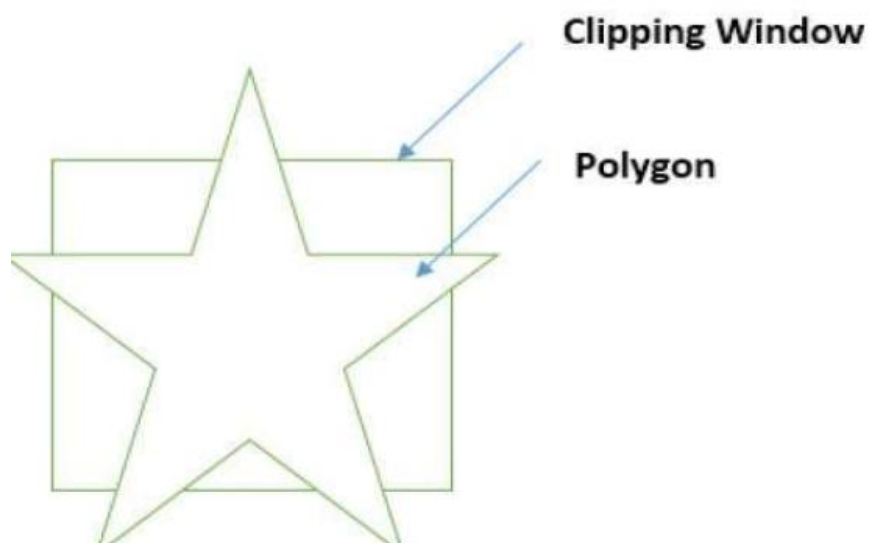
$t = \frac{N_i.[P_0 - P_{Ei}]}{N_i.D}$

It is valid for the following conditions –

- $N_i \neq 0$ (error cannot happen)
- $D \neq 0$ ($P_1 \neq P_0$)
- $N_i \cdot D \neq 0$ (P_0P_1 not parallel to E_i)

8.3 Polygon Clipping (Sutherland Hodgman Algorithm)

A polygon can also be clipped by specifying the clipping window. Sutherland Hodgeman polygon clipping algorithm is used for polygon clipping. In this algorithm, all the vertices of the polygon are clipped against each edge of the clipping window. First the polygon is clipped against the left edge of the polygon window to get new vertices of the polygon. These new vertices are used to clip the polygon against right edge, top edge, bottom edge, of the clipping window as shown in the following figure.



While processing an edge of a polygon with clipping window, an intersection point is found if edge is not completely inside clipping window and the partial edge from the intersection point to the outside edge is clipped. The following figures show left, right, top and bottom edge clippings.

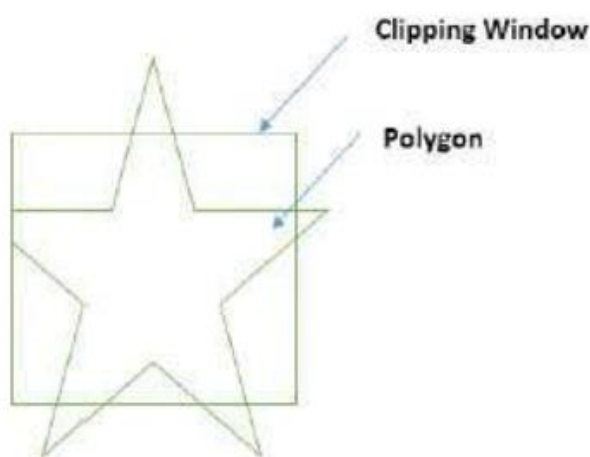


Figure: Clipping Left Edge

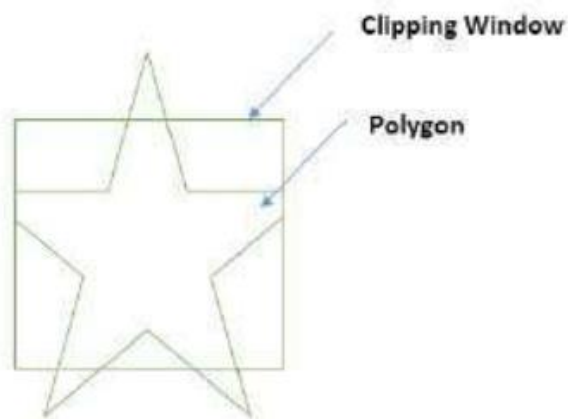


Figure: Clipping Right Edge

8.4 Text Clipping

Various techniques are used to provide text clipping in a computer graphics. It depends on the methods used to generate characters and the requirements of a particular application. There are three methods for text clipping which are listed below –

- **All or none string clipping**
- **All or none character clipping**

- Text clipping

The following figure shows all or none string clipping:

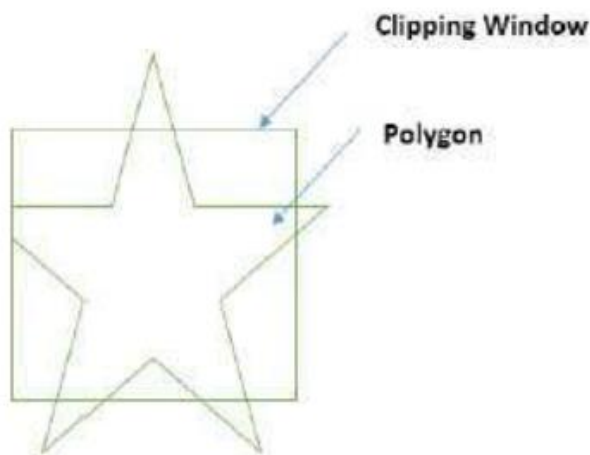


Figure: Clipping Left Edge

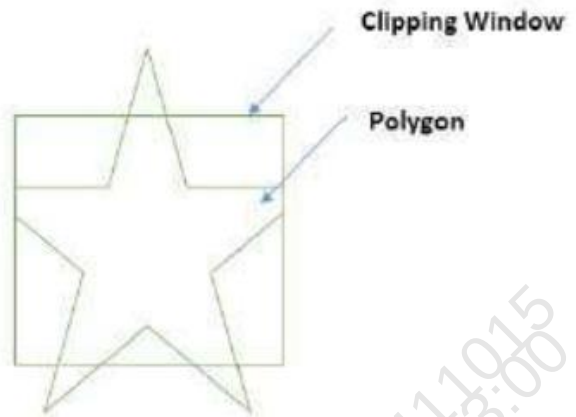
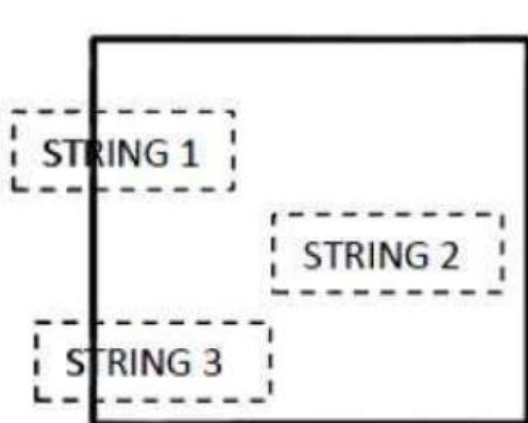
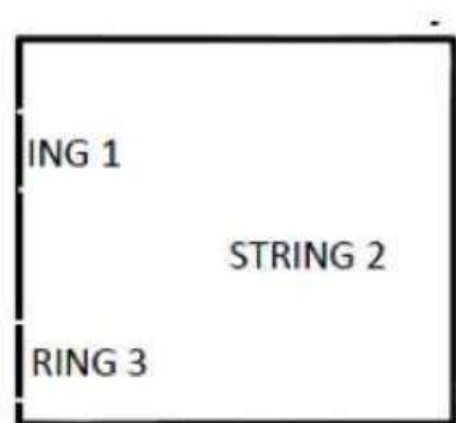


Figure: Clipping Right Edge

In all or none string clipping method, either we keep the entire string or we reject entire string based on the clipping window. As shown in the above figure, STRING2 is entirely inside the clipping window so we keep it and STRING1 being only partially inside the window, we reject. The following figure shows all or none character clipping:



Before Clipping



After Clipping

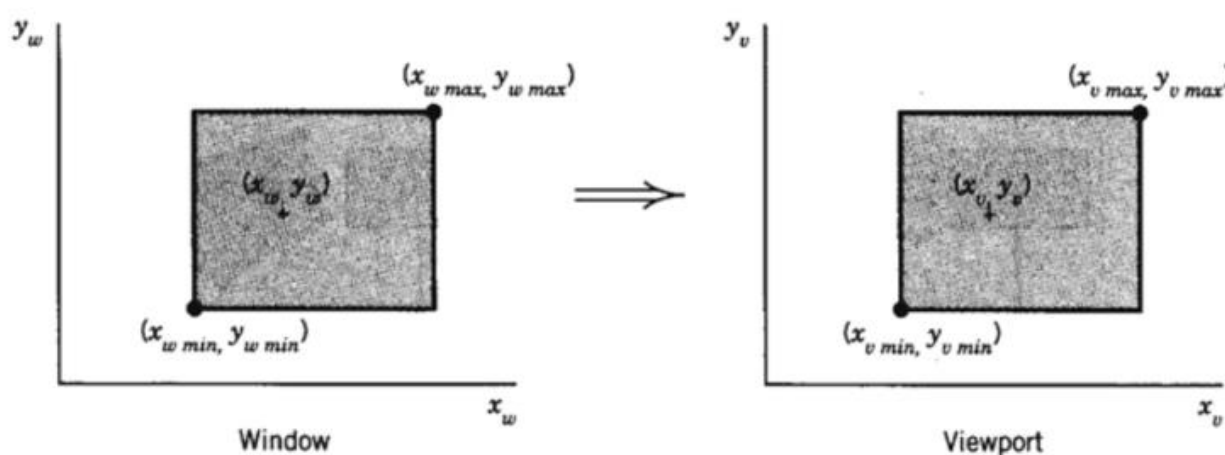
This clipping method is based on characters rather than entire string. In this method if the string is entirely inside the clipping window, then we keep it. If it is partially outside the window, then

- You reject only the portion of the string being outside
- If the character is on the boundary of the clipping window, then we discard only that portion of character that is outside of the clipping window.

5.6 Window port-to-Viewport mapping

In a typical application, we have a rectangle made of pixels, with its natural pixel coordinates, where an image will be displayed. This rectangle will be called the **viewport**. We also have a set of geometric objects that are defined in a possibly different coordinate system, generally one that uses real-number coordinates rather than integers. These objects make up the "scene" or "world" that we want to view, and the coordinates that we use to define the scene are called **world coordinates**.

For 2D graphics, the world lies in a plane. It's not possible to show a picture of the entire infinite plane. We need to pick some rectangular area in the plane to display in the image. Let's call that rectangular area the **window**, or view window. A coordinate transform is used to map the window to the viewport.



$$\frac{y_v - y_{vmin}}{y_{vmax} - y_{vmin}} = \frac{y_w - y_{wmin}}{y_{wmax} - y_{wmin}}$$

$$x_v = (x_w - x_{wmin}) \left(\frac{x_{vmax} - x_{vmin}}{x_{wmax} - x_{wmin}} \right) + x_{vmin}$$

$$y_v = (y_w - y_{wmin}) \left(\frac{y_{vmax} - y_{vmin}}{y_{wmax} - y_{wmin}} \right) + y_{vmin}$$

Any window can be specified by any four co-ordinates i.e. W_{xmin} , W_{xmax} , W_{ymin} and W_{ymax} and similarly a viewport can be represented by four normalized co-ordinates i.e. V_{xmin} , V_{xmax} , V_{ymin} and V_{ymax} viewing transformation of any point let $W(W_x, W_y)$, can be implemented using following equation. If normalized co-ordinates are $V(V_x, V_y)$.

then

$$\frac{W_x - W_{xmin}}{W_{xmax} - W_{xmin}} = \frac{V_x - V_{xmin}}{V_{xmax} - V_{xmin}}$$

$$\text{and} \quad \frac{W_y - W_{ymin}}{W_{ymax} - W_{ymin}} = \frac{V_y - V_{ymin}}{V_{ymax} - V_{ymin}}$$

$$\text{or,} \quad V_x = \frac{V_{xmax} - V_{xmin}}{W_{xmax} - W_{xmin}} (W_x - W_{xmin}) + V_{xmin}$$

$$V_y = \frac{V_{ymax} - V_{ymin}}{W_{ymax} - W_{ymin}} (W_y - W_{ymin}) + V_{ymin}$$

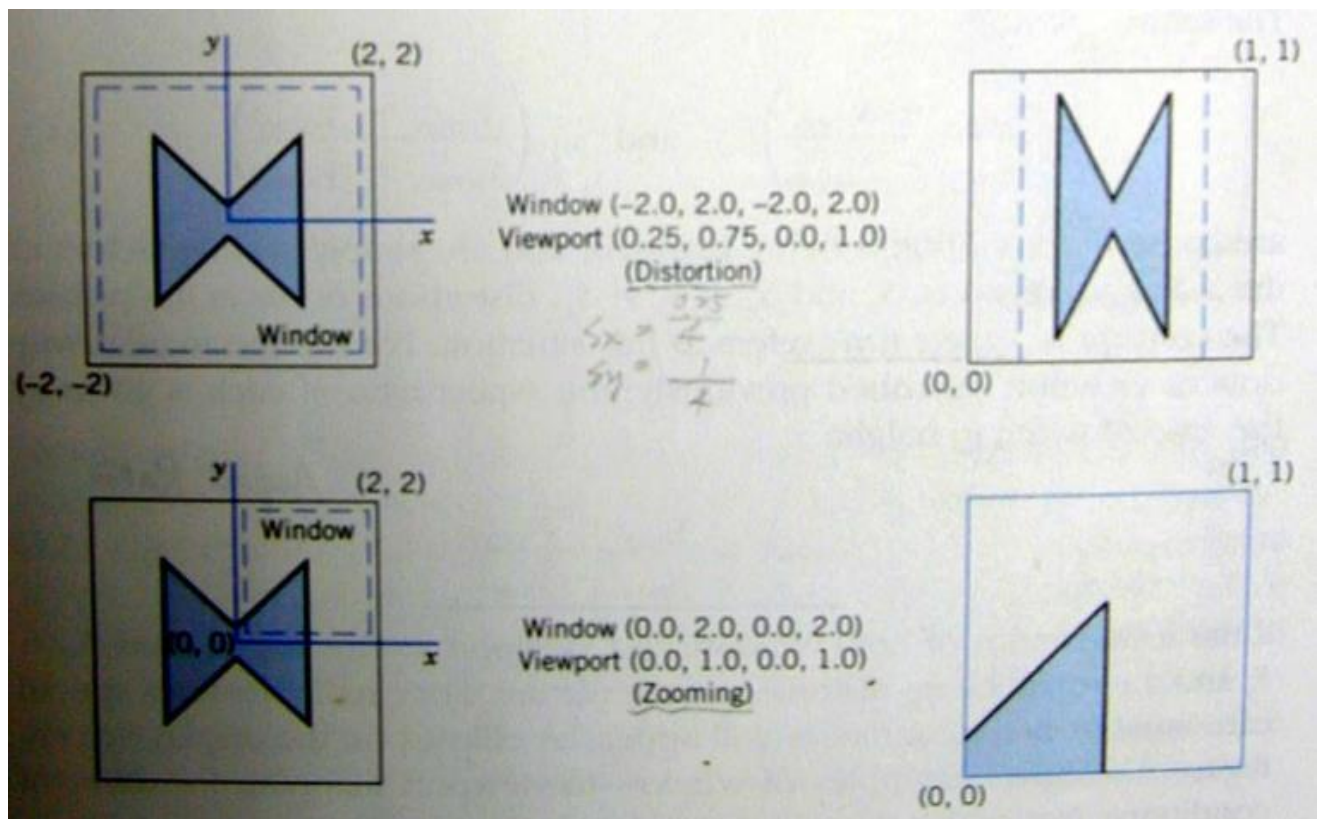
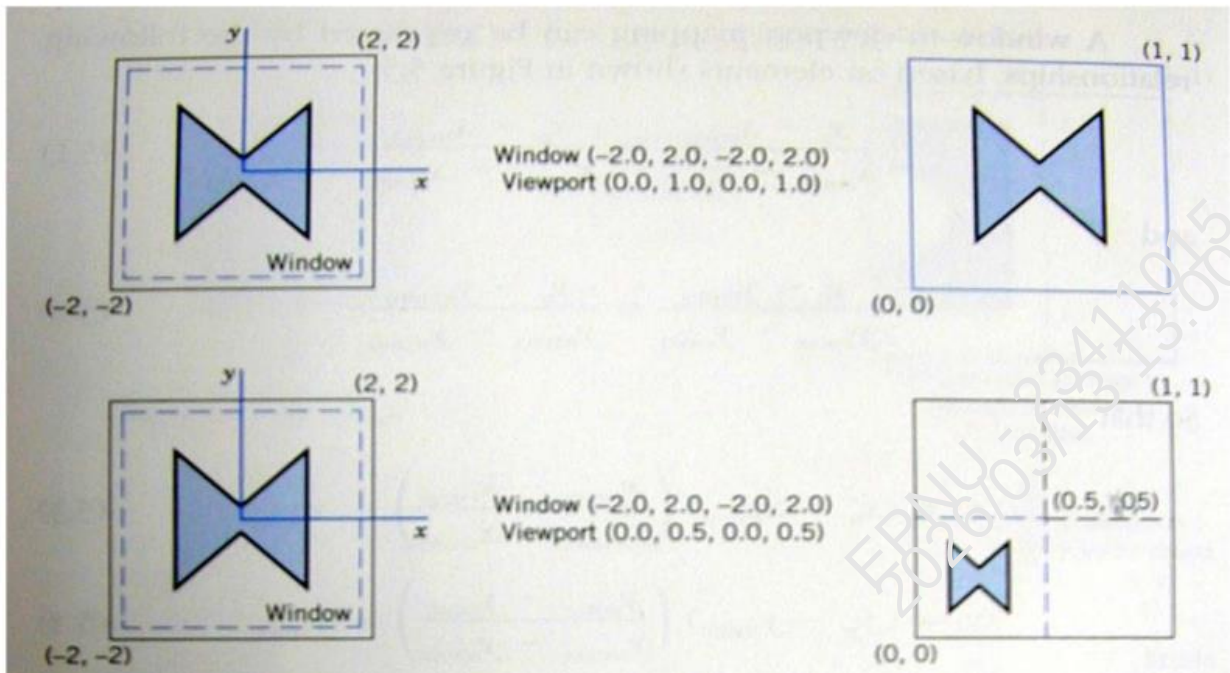
The viewing transformation can be represented in terms of basic transformation as above figure.

For rectangular window or viewport, the aspect ratio is given by the ratio of width to height

$$AR = \frac{x_{max} - x_{min}}{y_{max} - y_{min}}$$

Example:- Find the transformation matrix that will map points contained in a window whose lower left corner is at $(2, 2)$ and upper right corner is at $(6, 5)$ onto a normalized viewport that has a lower left corner at $(1/2, 1/2)$ and upper right corner at $(1, 1)$.

Solution :-



The Shear Transformation in each case :-

$$S_x = 1/4 \rightarrow 0.5/4 \rightarrow 0.5/4 \rightarrow 1/2$$

$$S_y = 1/4 \rightarrow 0.5/4 \rightarrow 1/4 \rightarrow 1/2$$

So that the distortion picture ($s_x=1/8$, $s_y=1/4$) in the third row is can be rewritten in matrix form,

$$[x_v, y_v, 1] = [x_w, y_w, 1] \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ (-s_x x_{wmin} + x_{vmin}) & (-s_y y_{wmin} + y_{vmin}) & 1 \end{bmatrix}$$

the transformation matrix becomes:

$$M_{map} = \begin{bmatrix} \frac{1}{8} & 0 & 0 \\ 0 & \frac{1}{6} & 0 \\ \frac{1}{4} & \frac{1}{6} & 1 \end{bmatrix}$$

EXERCISEs 2

- 1) Compare Bresenham line generation with DDA line generation.
- 2) Illustrate the Bresenham line generation algorithm by digitising the line with end points (15, 5) and (25,13).
- 3) Given a circle radius $r = 5$ determine positions along the circle octants in 1st Quadrant from $x = 0$ to $x = y$?
- 4) Distinguish between Scan line polygon fill and Seed fill or Flood fill algorithm?
- 5) A clipping window is given by $P_0(10,10)$, $P_1(20,10)$, $P_2(10,20)$, $P_3(20,20)$.
Using Cyrus Beck algorithm clip the line $A(0,0)$ and $B(15,25)$
- 6) Compare Cohen Sutherland Line clipping with Cyrus Beck Line clipping algorithm?
- 7) For a window $A(100, 10)$, $B(160, 10)$, $C(160, 40)$, $D(100, 40)$. Using Sutherland–Cohen clipping algorithm find the visible portion of the line segments EF, GH and P1P2. $E(50,0)$, $F(70,80)$, $G(120, 20)$, $H(140, 80)$, $P_1(120, 5)$, $P_2(180, 30)$.

Chapter 3

Geometrical Transformations

1. Two Dimensional Transformations

1.1 Translation

1.2 Rotation

1.3 Scaling

1.4 Shearing

1.5 Reflection

2. Composite Transformations

3. Three Dimensional Transformations

4.1 Transformations for 3D Translation

4.2 Transformations for 3D Scaling

4.3 Transformations for 3D Rotation

4.4 Transformations for 3D Shearing

5. Co-ordinate Transformations

OBJECTIVE

The objects are referenced by their coordinates. Changes in orientation, size and shape are accomplished with geometric transformations that allow us to calculate the new coordinates. After going through this unit, you should be able to:

- Describe the basic transformations for 2-D translation, rotation, scaling and shearing;
- Discuss the role of composite transformations
- Describe composite transformations for Rotation about a point and reflection about a line;
- Define and explain the use of homogeneous coordinate systems for the transformations
- Describe the 3-D transformations of translation, rotation, scaling and shearing;

1. Two Dimensional Transformations

Transformations are also used to generate surfaces and solids by sweeping curves and surfaces, respectively. The term 'sweeping' refers to parametric transformations, which are utilized to generate surfaces and solids. When we sweep a curve, it is transformed through several positions along or around an axis, generating a surface.

Transformation is the backbone of computer graphics, enabling us to manipulate the shape, size, and location of the object. It can be used to effect the following changes in a geometric object:

- Change the location
- Change the Shape
- Change the size
- Rotate
- Copy
- Generate a surface from a line
- Generate a solid from a surface
- Animate the object

The appearance of the generated surface depends on the number of instances of the transformation.

Basically there are two categories of transformation:–

- a) Geometric Transformation
- b) Co-ordinate Transformation

a) Geometric Transformation: – Every object is assumed to be a set of points. Every object point P is denoted by the ordered pairs known as coordinates (x, y) and so the

object is the sum of total of all the coordinates points. If any object is transform to a new position, then the coordinates of new position can be obtained by the application of Geometric Transformation.

- b) Co-ordinate Transformation:** – When object itself relative to a stationary co-ordinate system or background, referred as geometric transformations and applied to each point of the object. And while the co-ordinate system is moved relative to the object and object is held stationary then this process is termed as co-ordinate transformation.
- c)** Let us look at the procedure for carrying out basic transformations, which are based on matrix operation. A transformation can be expressed as $[P^*] = [P] [T]$ where, $[P^*]$ is the new coordinates matrix $[P]$ is the original coordinates matrix, or points matrix $[T]$ is the transformation matrix.

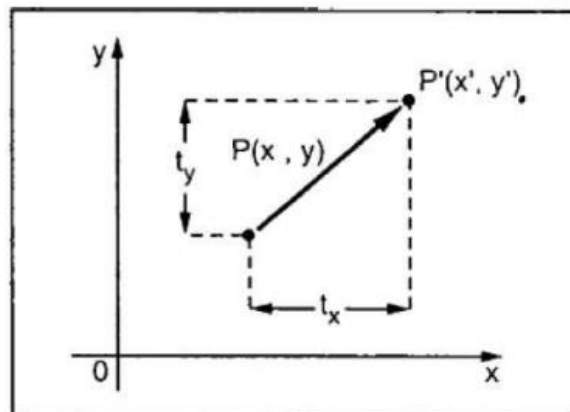
With the z-terms set to zero, the P matrix can be written as, $[P] =$

$$[P] = \begin{bmatrix} x1 & y1 & 0 \\ x2 & y2 & 0 \\ \vdots & \vdots & \vdots \\ xn & yn & 0 \end{bmatrix}$$

Values of the elements in the matrix will change according to the type of transformation being used, as we will see shortly. The transformation matrix changes the size, position, and orientation of an object, by mathematically adding, or multiplying its coordinate values.

1.1 Translation

A translation moves an object to a different position on the screen. You can translate a point in 2D by adding translation coordinate (t_x, t_y) to the original coordinate (X, Y) to get the new coordinate (X', Y') .



From the above figure, you can write that –

$$X' = X + t_x$$

$$Y' = Y + t_y$$

In the matrix form

$$T_V = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

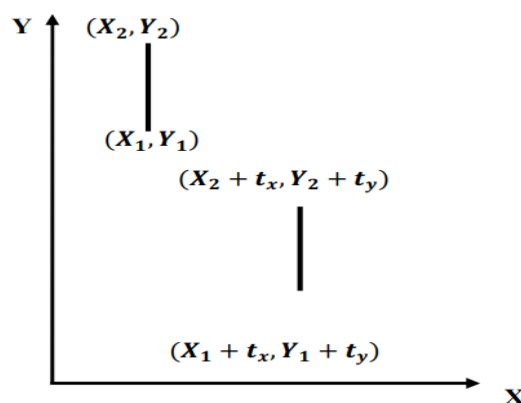
The pair (t_x, t_y) is called the translation vector or shift vector.

If (x, y) is the original point and (x_1, y_1) is the transformed point, then the formula for a translation **is**

$$x_1 = x + e$$

$$y_1 = y + f$$

where e is the number of units by which the point is moved horizontally and f is the amount by which it is moved vertically.



Translation of a line in X and Y-direction

Example1: Translate a square ABCD with the coordinates

A(0,0), B(5,0), C(5,5), D(0,5) by 2 units in x-direction and 3 units in y-direction.

Hint : We can represent the given square, in matrix form, using homogeneous coordinates of vertices as :-

$$\begin{matrix} x1 & y1 & 1 & 0 & 0 & 1 \\ x2 & y2 & 1 & 5 & 0 & 1 \\ x3 & y3 & 1 & 5 & 5 & 1 \end{matrix}$$

The translation factors are, $t_x=2$, $t_y=3$

The transformation matrix for translation :

$$T_x = \begin{matrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ t_x & t_y & 1 \end{matrix} = \begin{matrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 2 & 3 & 1 \end{matrix}$$

New object point coordinates are:

$$[A'B'C'D'] = [ABCD].T_x$$

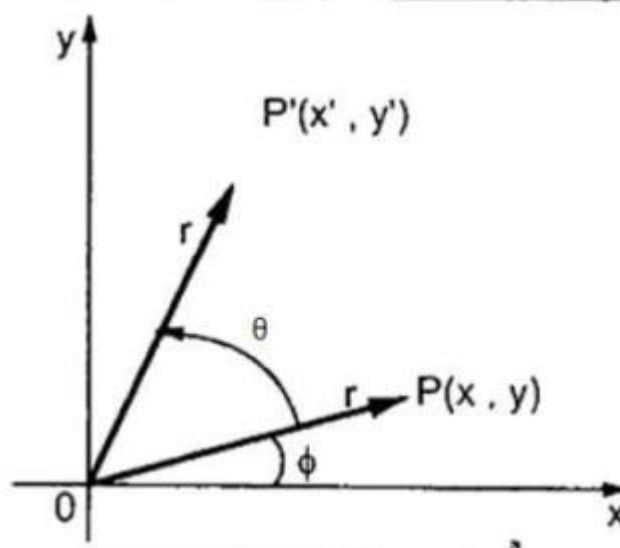
$$\begin{matrix} A' \\ B' \\ C' \\ D' \end{matrix} \begin{pmatrix} x'_1 & y'_1 & 1 \\ x'_2 & y'_2 & 1 \\ x'_3 & y'_3 & 1 \\ x'_4 & y'_4 & 1 \end{pmatrix} = \begin{pmatrix} 0 & 0 & 1 \\ 5 & 0 & 1 \\ 5 & 5 & 1 \\ 0 & 5 & 1 \end{pmatrix} \cdot \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 2 & 3 & 1 \end{pmatrix}$$

$$= \begin{pmatrix} 2 & 3 & 1 \\ 7 & 3 & 1 \\ 7 & 8 & 1 \\ 2 & 8 & 1 \end{pmatrix}$$

1.2 Rotation

In rotation, we rotate the object at particular angle θ (theta) from its origin. From the following figure, we can see that the point $P(X, Y)$ is located at angle ϕ from the horizontal X coordinate with distance r from the origin.

Let us suppose you want to rotate it at the angle θ . After rotating it to a new location, you will get a new point $P'(X', Y')$.



Representing the above equation in matrix form for **Anti-clockwise** direction,

$$R_{\theta} = \begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix}$$

$$\begin{bmatrix} x' & y' \end{bmatrix} = \begin{bmatrix} x & y \end{bmatrix} \begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix}$$

Where R is the rotation matrix

Representing the above equation in matrix form for **clockwise** direction,

$$= \begin{pmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Example 2: Perform a 45° rotation of a triangle A(0,0),B(1,1),C(5,2) about the origin.

Anti-clockwise

Solution: We can represent the given triangle, in matrix form, using homogeneous coordinates of the vertices:

$$\begin{bmatrix} 0 & 0 & 1 \\ 1 & 1 & 1 \\ 5 & 2 & 1 \end{bmatrix}$$

$$\text{The matrix of rotation is: } R_{\theta} = R_{45} = \begin{bmatrix} \cos 45 & \sin 45 & 0 \\ -\sin 45 & \cos 45 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} & 0 \\ -\frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

So the new coordinates A'B'C' of the rotated triangle ABC can be found as:

$$[A'B'C'] = [ABC] \cdot R_{45^\circ} = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 1 & 1 \\ 5 & 2 & 1 \end{bmatrix} \begin{bmatrix} \sqrt{2}/2 & \sqrt{2}/2 & 0 \\ -\sqrt{2}/2 & \sqrt{2}/2 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 \\ 0 & \sqrt{2} & 1 \\ 3\sqrt{2}/2 & 7\sqrt{2}/2 & 1 \end{bmatrix}$$

1.3 Scaling

In scaling transformation, the original coordinates of an object are multiplied by the given scale factor. There are two types of scaling transformations: uniform and non-uniform. In the uniform scaling, the coordinate values change uniformly along the x, y, and z coordinates, whereas, in non-uniform scaling, the change is not necessarily the same in all the coordinate directions.

The mathematical expression for pure scaling is

$$x' = S_x \cdot x$$

$$y' = S_y \cdot y$$

Matrix equation of a non-uniform scaling has the form:

$$\Rightarrow \begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} S_x & 0 \\ 0 & S_y \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

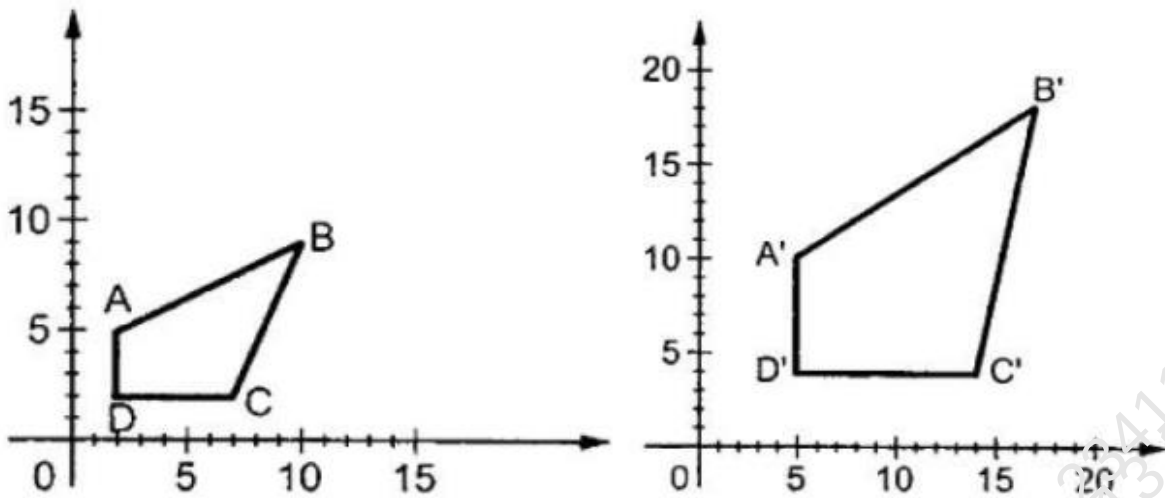
where, s_x , s_y are the scale factors for the x and y coordinates of the object. That is, if a point (x, y) is scaled by a factor of **a** in the x direction and by a factor of **d** in the y direction, then the resulting point (x_1, y_1) is given by:

$$x_1 = a * x$$

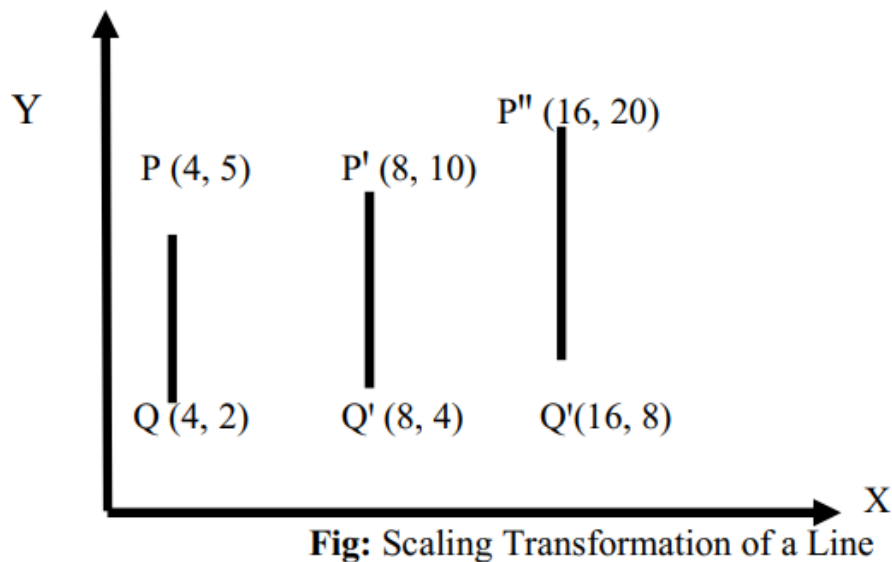
$$y_1 = d * y$$

If you apply this transform to a shape that is centered at the origin, it will stretch the shape by a factor of **a** horizontally and **d** vertically.

Where S is the scaling matrix. The scaling process is shown in the following figure.



If we provide values less than 1 to the scaling factor S , then we can reduce the size of the object. If we provide values greater than 1, then we can increase the size of the object.



Line PQ be scaled to P'Q' when $S_x=S_y=0.5$ and to P''Q'' when $S_x=S_y=2$

Obviously,

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \frac{1}{2} & 0 \\ 0 & \frac{1}{2} \end{bmatrix} \begin{bmatrix} 8 \\ 4 \end{bmatrix} = \begin{bmatrix} 4 \\ 2 \end{bmatrix}$$

Thus $Q' = (4, 2)$

Similarly $P' = (4, 5)$

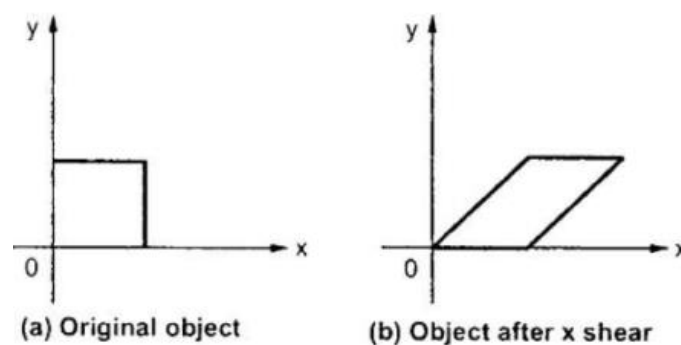
when $S_x = S_y = 0.5$

1.4 Shearing

A transformation that slants the shape of an object is called the shear transformation. There are two shear transformations **X-Shear** and **Y-Shear**. One shifts X coordinates values and other shifts Y coordinate values. However; in both the cases only one coordinate changes its coordinates and other preserves its values. Shearing is also termed as **Skewing**.

X-Shear

The X-Shear preserves the Y coordinate and changes are made to X coordinates, which causes the vertical lines to tilt right or left as shown in below figure.



The transformation matrix for X-Shear about the origin by **a** distance can be represented as –

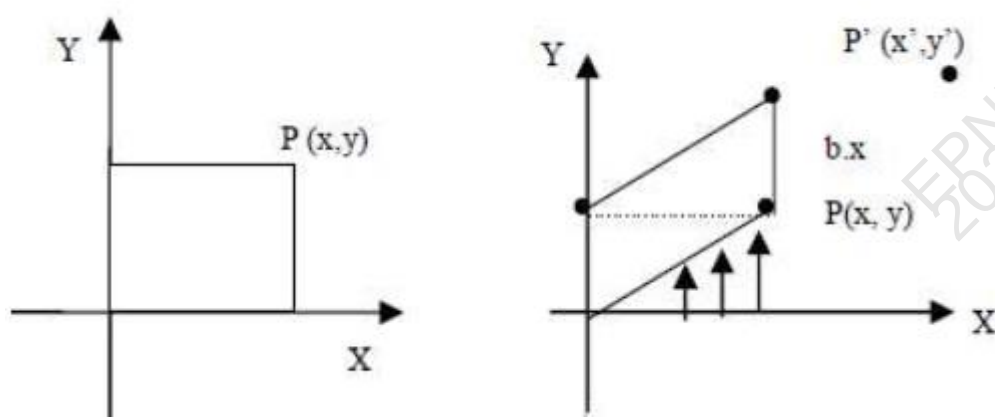
$$\begin{bmatrix} 1 & 0 & 0 \\ a & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

$$X' = X + Sh_x \cdot Y$$

$$Y' = Y$$

Y-Shear

The Y-Shear preserves the X coordinates and changes the Y coordinates which causes the horizontal lines to transform into lines which slopes up or down as shown in the following figure.



The Y-Shear can be represented in matrix form as –

$$\begin{bmatrix} 1 & b & 0 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

$$Y' = Y + Sh_y \cdot X$$

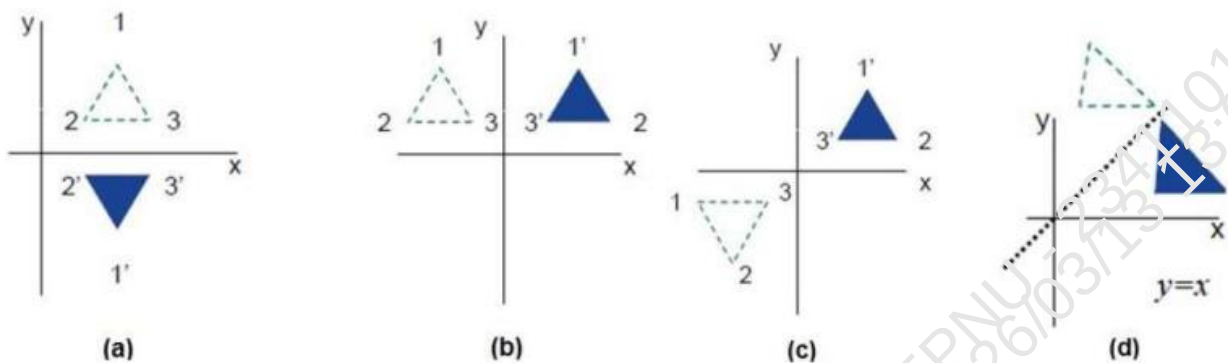
$$X' = X$$

Although shears can in fact be built up out of rotations and scalings if necessary, it is not really obvious how to do so. A shear will "tilt" objects. A horizontal shear will tilt things towards the left (for negative shear) or right (for positive shear). A vertical shear tilts them up or down.

1.5 Reflection

Reflection is the mirror image of original object. In other words, we can say that it is a rotation operation with 180° . In reflection transformation, the size of the object does not change.

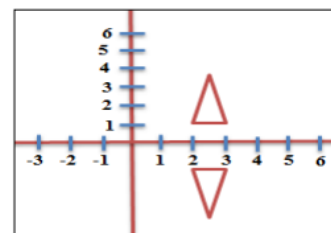
The following figures show reflections with respect to X and Y axes, and about the origin respectively.



1- Reflection on the **X-axis**

$$\begin{aligned} X_{\text{new}} &= X \\ Y_{\text{new}} &= -Y \end{aligned} \quad \text{OR}$$

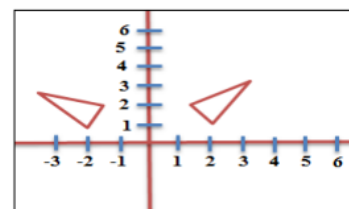
1	0	0
0	-1	0
0	0	1



2- Reflection on the **Y-axis**

$$\begin{aligned} X_{\text{new}} &= -X \\ Y_{\text{new}} &= Y \end{aligned} \quad \text{OR}$$

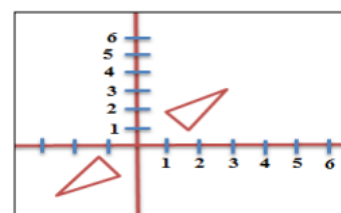
-1	0	0
0	1	0
0	0	1



3- Reflection on the **origin**

$$\begin{aligned} X_{\text{new}} &= -X \\ Y_{\text{new}} &= -Y \end{aligned} \quad \text{OR}$$

-1	0	0
0	-1	0
0	0	1

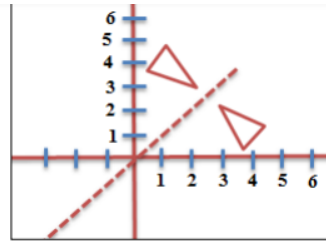


4- Reflection on the line $Y = X$

$$\begin{aligned} X_{\text{new}} &= Y \\ Y_{\text{new}} &= X \end{aligned}$$

OR

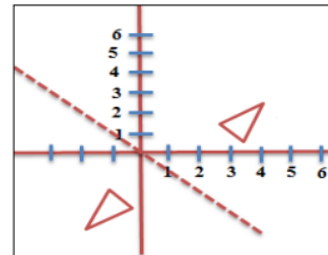
0	1	0
1	0	0
0	0	1

**5- Reflection on the line $Y = -X$**

$$\begin{aligned} X_{\text{new}} &= -Y \\ Y_{\text{new}} &= -X \end{aligned}$$

OR

0	-1	0
-1	0	0
0	0	1

**Example 1**

Reflect the point P(3,2) in : A- X axis; B- Y axis; C- origin; D-line $Y=X$;

Solution:**A- X axis**

$$\begin{bmatrix} X1 \text{ new} & Y1 \text{ new} & 1 \end{bmatrix} = \begin{bmatrix} 3 & 2 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 3 & -2 & 1 \end{bmatrix}$$

B- Y axis

$$\begin{bmatrix} X1 \text{ new} & Y1 \text{ new} & 1 \end{bmatrix} = \begin{bmatrix} 3 & 2 & 1 \end{bmatrix} * \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} -3 & 2 & 1 \end{bmatrix}$$

C- Origin

$$\begin{bmatrix} X1 \text{ new} & Y1 \text{ new} & 1 \end{bmatrix} = \begin{bmatrix} 3 & 2 & 1 \end{bmatrix} * \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} -3 & -2 & 1 \end{bmatrix}$$

D- Line $Y=X$

$$\begin{bmatrix} X1 \text{ new} & Y1 \text{ new} & 1 \end{bmatrix} = \begin{bmatrix} 3 & 2 & 1 \end{bmatrix} * \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 3 & 1 \end{bmatrix}$$

Example 2 :

Reflect the triangle with vertices at A(2,4), B(4,6), C(2,6) in : A- X axis

Solution 1:

$$X_{\text{new}} = X$$

$$Y_{\text{new}} = -Y$$

Point (2,4)

$$X1_{\text{new}} = 2$$

$$Y1_{\text{new}} = -4$$

Point (4,6)

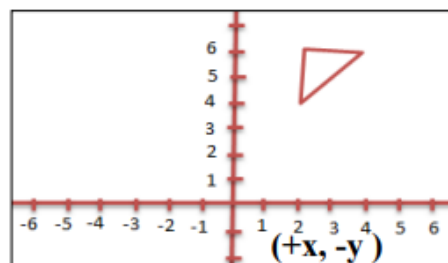
$$X1_{\text{new}} = 4$$

$$Y1_{\text{new}} = -6$$

Point (2,6)

$$X1_{\text{new}} = 2$$

$$Y1_{\text{new}} = -6$$

**Solution 2:**

X1 new	Y1 new	1
X2 new	Y2 new	1
X3 new	Y3 new	1

=

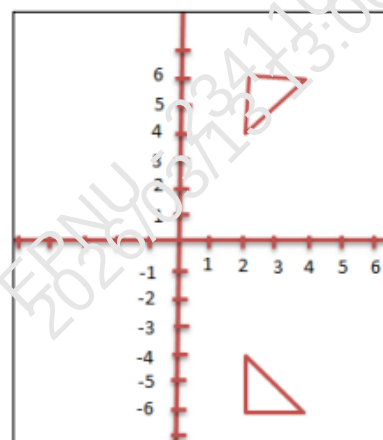
2	4	1
4	6	1
2	6	1

*

1	0	0
0	-1	0
0	0	1

=

2	-4	1
4	-6	1
2	-6	1



Example 3 :

Reflect the triangle with vertices at A(2,4), B(4,6), C(2,6) in : Y axis;

Solution 1:

$$X_{\text{new}} = -X$$

$$Y_{\text{new}} = Y$$

Point (2,4)

$$X1_{\text{new}} = -2$$

$$Y1_{\text{new}} = 4$$

Point (4,6)

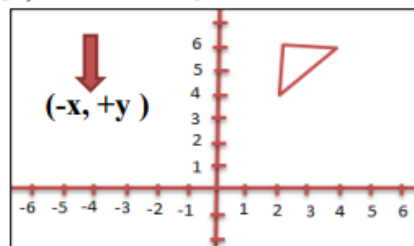
$$X1_{\text{new}} = -4$$

$$Y1_{\text{new}} = 6$$

Point (2,6)

$$X1_{\text{new}} = -2$$

$$Y1_{\text{new}} = 6$$

**Solution 2:**

X1 new	Y1 new	1
X2 new	Y2 new	1
X3 new	Y3 new	1

=

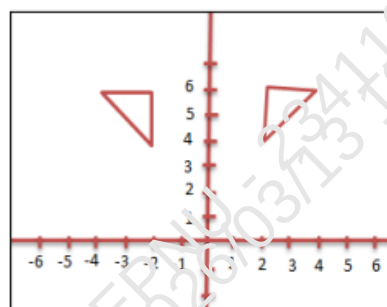
2	4	1
4	6	1
2	6	1

*

-1	0	0
0	1	0
0	0	1

=

-2	4	1
-4	6	1
-2	6	1



2. Composite Transformation

We have seen the basic matrix transformations for translation, rotation, reflection, scaling and shearing with respect to the origin of the coordinate system. By multiplying these basic matrix transformations, we can build complex transformations, such as rotation about an arbitrary point, mirror reflection about a line etc. This process is called concatenation of matrices and the resulting matrix is often referred to as the composite transformation matrix. Inverse transformations play an important role when you are dealing with composite transformation. They come to the rescue of basic transformations by making them applicable during the construction of composite transformation. You can observed that the Inverse transformations for translation, rotation, reflection, scaling and shearing have the following relations, and v , θ , a , b , s_x , s_y , s_z are all parameter involved in the transformations. If a transformation of the plane T_1 is

followed by a second plane transformation T_2 , then the result itself may be represented by a single transformation T which is the composition of T_1 and T_2 taken in that order. This is written as $T = T_1 \cdot T_2$.

Composite transformation can be achieved by concatenation of transformation matrices to obtain a combined transformation matrix.

A combined matrix – $[T][X] = [X] [T_1] [T_2] [T_3] [T_4] \dots [T_n]$

Where $[T_i]$ is any combination of

- Translation
- Scaling
- Shearing
- Rotation
- Reflection

The change in the order of transformation would lead to different results, as in general matrix multiplication is not cumulative, that is $[A] \cdot [B] \neq [B] \cdot [A]$ and the order of multiplication.

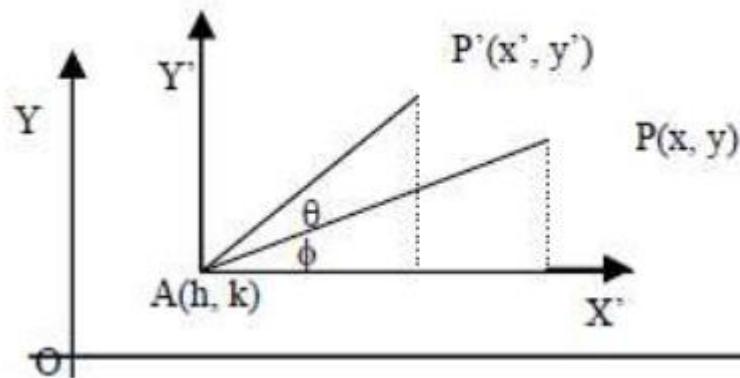
The basic purpose of composing transformations is to gain efficiency by applying a single composed transformation to a point, rather than applying a series of transformation, one after another.

For example, to rotate an object about an arbitrary point (X_p, Y_p) , we have to carry out three steps:

- Translate point (X_p, Y_p) to the origin.
- Rotate it about the origin.
- Finally, translate the center of rotation back where it belonged

Example: Given a 2-D point $P(x,y)$, which we want to rotate, with respect to an arbitrary point $A(h,k)$.

Let $P'(x'y')$ be the result of anticlockwise rotation of point P by angle θ about A



Since, the rotation matrix R_θ is defined only with respect to the origin, we need a set of basic transformations, which constitutes the composite transformation to compute the rotation about a given arbitrary point A, denoted by $R_{\theta,A}$. We can determine the transformation $R_{\theta,A}$ in three steps:

- 1) Translate the point A(h,k) to the origin O, so that the center of rotation A is at the origin.
- 2) Perform the required rotation of θ degrees about the origin, and
- 3) Translate the origin back to the original position A(h,k)

$$R_{\theta,A} = T_{-v} \cdot R_\theta \cdot T_v$$

$$= \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -h & -k & 1 \end{pmatrix} \begin{pmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ h & k & 1 \end{pmatrix}$$

$$= \begin{pmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ (1-\cos\theta).h+k.\sin\theta & (1-\cos\theta).k-h.\sin\theta & 1 \end{pmatrix}.$$

Example: Perform a 45° rotation of a triangle A (0,0), B (1,1), C (5,2) about an arbitrary point P(-1, -1).

Solution: Given triangle ABC, as show in Figure (a), can be represented in homogeneous coordinates of vertices as:

$$[A \ B \ C] = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 1 & 1 \\ 5 & 2 & 1 \end{bmatrix}$$

A rotation matrix R_Q , about a given arbitrary point A (h, k) is:

$$\begin{bmatrix} \cos \theta & \sin \theta & 1 \\ -\sin \theta & \cos \theta & 1 \\ (1 - \cos \theta) \cdot j + k \cdot \sin \theta & (1 - \cos \theta) \cdot k - j \cdot \sin \theta & 1 \end{bmatrix}$$

3. Three Dimensional Transformations

Transformations in 3 dimensional space are analogous to those in two dimensional space in many ways. Transformation matrix is the basic tool for 3-D transformation as was for 2-D. A matrix with n x m dimensions is multiplied with the coordinate of objects. Usually 3 x 3 or 4 x 4 matrices are used for transformation.

Transformations for 3-D Translation

In a three dimensional homogeneous co-ordinate representation a point is translated from position $P(x, y, z)$ to position $P'(x', y', z')$ with the matrix operation.

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

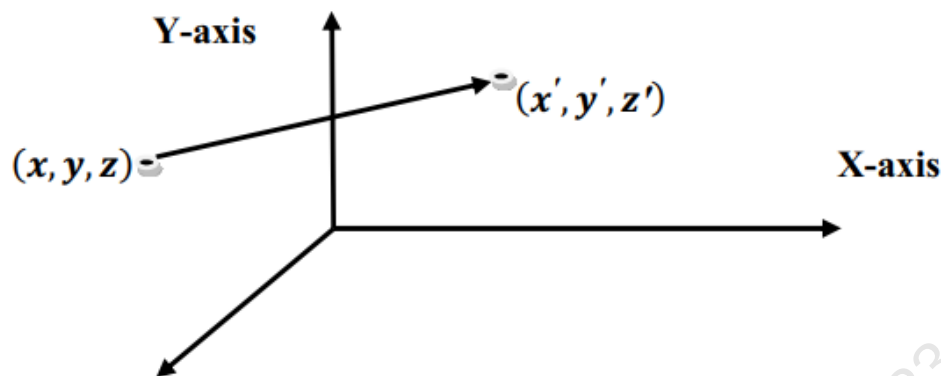
Or $P' = T \cdot P$

Parameters t_x , t_y and t_z specifying translation distances for the co-ordinate directions x, y and z are assigned any real values. The above matrix representation is equivalent to the three equations.

$$x' = x + t_x$$

$$y' = y + t_y$$

$$z' = z + t_z$$



An object is translated in three dimensions by transforming each of the defining points of the object. For an object represented as a set of polygon surfaces, we translate each vertex of each surface and redraw the polygon facets in the new position. We obtain the inverse translation matrix in the above matrix by negating the translation distances t_x , t_y and t_z . This produces a translation in the opposite direction and the product of a translation matrix and its inverse produces the identity matrix.

3.1 Transformations for 3-D Scaling

The process of compressing and expanding any object is called **Scaling**. Thus scaling alters the size of an object if scale factors. Scaling can be achieved by multiplying the original coordinates of the object with the scaling factor to get the desired result.

$S_x = S_y = S_z = S > 1$ The scaling is called magnification.

And if $S_x = S_y = S_z = S < 1$ the scaling is termed as reduction.

The points after scaling with respect to origin can be calculated by the following relation

$$P'(x', y', z') = S \cdot P(x, y, z)$$

$$x' = S_x \cdot x$$

$$y' = S_y \cdot y$$

$$z' = S_z \cdot z$$

The equation can be written in the matrix form as: –

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} S_x & 0 & 0 & a \\ 0 & S_y & 0 & b \\ 0 & 0 & S_z & c \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Where

$$S = \begin{bmatrix} S_x & 0 & 0 & 0 \\ 0 & S_y & 0 & 0 \\ 0 & 0 & S_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Scaling with respect to a selected fixed position (x_f, y_f, z_f) can be represented with the following transformation sequence:

- (1) Translation of the fixed point to the origin.
- (2) Scaling of the object related to the coordinate origin.
- (3) Translation of the fixed point back to its original position.

$$T(x_f, y_f, z_f) \cdot S(S_x, S_y, S_z) \cdot T(-x_f, -y_f, -z_f)$$

$$= \begin{bmatrix} S_x & 0 & 0 & (1 - S_x)x_f \\ 0 & S_y & 0 & (1 - S_y)y_f \\ 0 & 0 & S_z & (1 - S_z)z_f \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The following figure shows the effect of 3D scaling.

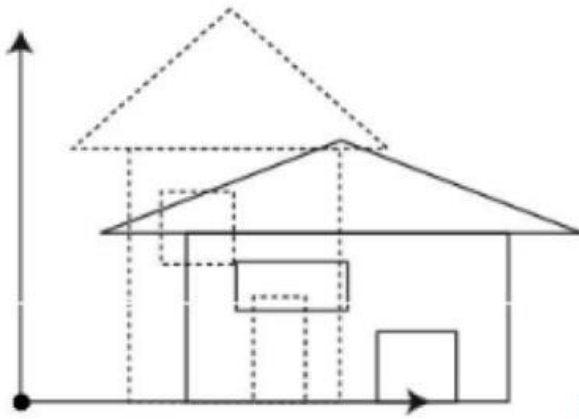
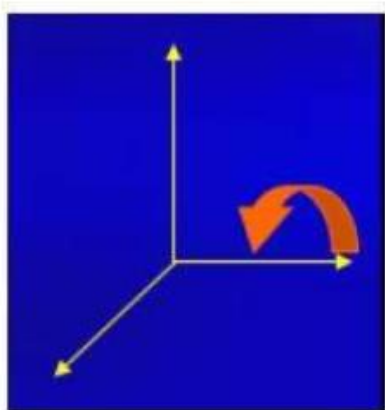


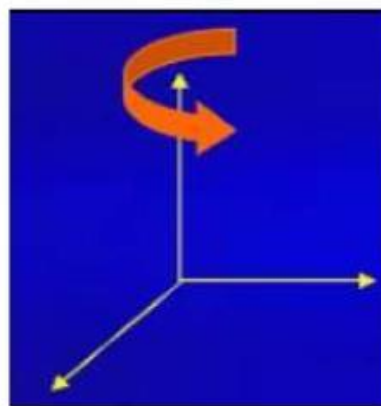
Fig :- Three Dimensional Scaling

3.2 Transformations for 3-D Rotation

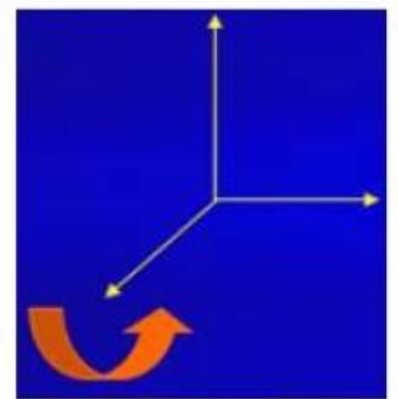
Rotation in three dimensional geometry is more complex than that of two dimensional, because in this case we consider the angle of rotation and axis of rotation. Therefore, there may be three cases, when we choose one of the positive x - axis , y - axis, z - axis , as an axis of rotation.



Rotation about x-axis



Rotation about y-axis



Rotation about z-axis

a. **Rotation about x-axis (i.e. axis of rotation is x):**

From the figure : $x' = x$

$$y' = r \cos(\alpha + \theta) = r \cos \alpha \cdot \cos \theta - r \sin \alpha \cdot \sin \theta$$

$$\text{since } r \cos \alpha = y$$

$$\text{and } r \sin \alpha = z$$

$$\text{or, } y' = y \cos \theta - z \sin \theta$$

$$\begin{aligned} z' = r \sin(\alpha + \theta) &= r \sin \alpha \cdot \cos \theta + r \cos \alpha \cdot \sin \theta \\ &= z \cos \theta + y \sin \theta \end{aligned}$$

$$x' = x$$

$$\text{or } R_{\theta,i} = y' = y \cos \theta - z \sin \theta$$

$$z' = y \sin \theta + z \cos \theta$$

The points after rotation can be calculated by the equation

$$P'(x', y', z') = R_{\theta,i} \cdot P(x, y, z)$$

When x' , y' and z' are defined by the above equation. The above equation can be written into matrix form as follows.

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

b. Rotation about y-axis (i.e. y-axis is the axis of rotation): –

$$x' = x \cos \theta + z \sin \theta$$

Similarly

$$y' = y$$

$$z' = -x \sin \theta + z \cos \theta$$

$$R_{\theta,i} = \begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ \sin \theta & 0 & \cos \theta \end{bmatrix}$$

And the equation in the matrix form,

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

3.3 Transformations for 3-D Shearing

Shear along any pair of axes is proportional to the third axes for instance, to shear along Z in **3D**, **x** and **y** values are altered by an amount proportional to the value of Z, leaving unchanged. Let Sh_{xy} , Sh_{zy} is the shear due to **Z** along **x** and **y** directions respectively and are real values. Then the matrix representation is

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ Sh_{zx} & Sh_{zy} & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Shear for **x**, **y** axis is similar to that of **Z**. The general form of shear is given by

$$\begin{bmatrix} 1 & Sh_{xy} & Sh_{xz} & 0 \\ 0 & 1 & Sh_{yz} & 0 \\ Sh_{yx} & Sh_{zy} & 1 & 0 \\ Sh_{zx} & 0 & 0 & 1 \end{bmatrix}$$

In 2- D shear transformation we discussed shear transformations relative to the x or, y-axis to produce distortions in the shapes of planer objects. In the 3D we can also

generate shears relative to z-axis and the result is change of volume and the 3-D shape of any object.

Example for shear along Z- axis

$$x' = x + az$$

$$y' = y + bz$$

$$z' = z$$

The corresponding transformation matrix

$$[T_{sz}] = \begin{bmatrix} 1 & 0 & a & 0 \\ 0 & 1 & b & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix}$$

The effect of this transformation matrix is to alter **x** and **y** coordinate values by an amount that is proportional to **z** value, while leaving the **z**-coordinate unchanged. 3D shearing transformation can be carried out about the other two principal axes as well.

An X-axis 3D shear can be expressed as,

$$x' = x$$

$$y' = y + ax$$

$$z' = z + bx$$

The corresponding transformation matrix is

$$[T_{sx}] = \begin{bmatrix} 1 & 0 & 0 & 0 \\ a & 1 & 0 & 0 \\ b & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Similarly Y-axis 3D shear can be expressed as,

$$x' = x + y$$

$$y' = y$$

$$z' = z + by$$

The corresponding transformation matrix is,

$$[T_{sy}] = \begin{bmatrix} 1 & a & 0 & 0 \\ a & 1 & 0 & 0 \\ b & b & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

EXERCISEs

- 1) A square consists of vertices A(0,0),B(0,1),C(1,1),D(1,0). After the translation C is found to be at the new location (6,7). Determine the new location of other vertices.
- 2) A square ABCD is given with vertices A(0,0),B(1,0),C(1,1), and D(0,1). Illustrate the effect of:
 - a) x-shear
 - b) y-shear
 - c) xy-shear on the given square, when a=2 and b=3
- 3) Find the new coordinates of a triangle A(0,0),B(1,1),C(5,2) after it has been (a) magnified to twice its size and (b) reduced to half its size.
- 4) A point P(3,3) makes a rotating of 450 about the origin and then translating in the direction of vector $v=5I+6J$. Find the new location of P.
- 5) Show that the order in which transformations are performed is important by applying the transformation of the triangle ABC by:

- (i) Rotating by 45° about the origin and then translating in the direction of the vector $(1,0)$, and
- (ii) Translating first in the direction of the vector $(1,0)$, and then rotating by 45° about the origin, where $A = (1, 0)$ $B = (0, 1)$ and $C = (1, 1)$.

6) Give a single 3×3 homogeneous coordinate transformation matrix, which will have the same effect as each of the following transformation sequences:

- a) Scale the image to be twice as large and then translate it 1 unit to the left.
- b) Scale the x direction to be one-half as large and then rotate counterclockwise by 90° about the origin.
- c) Rotate counterclockwise about the origin by 90° and then scale the x direction to be one-half as large.
- d) Translate down $\frac{1}{2}$ unit, right $\frac{1}{2}$ unit, and then rotate counterclockwise by 45° .

Chapter 4

Practical

Suppose that we want to draw a line starting at pixel $\mathbf{p}_s = (x_s, y_s)$ and ending at pixel $\mathbf{p}_e = (x_e, y_e)$ in the first octant (Figure). If we let $s = (y_e - y_s)/(x_e - x_s)$ be the slope of the line, then the pixel sequence we select can be derived from the explicit line equation

$$y = y_s + \text{round}(s(x - x_s));$$

$$x = x_s, \dots, x_e$$

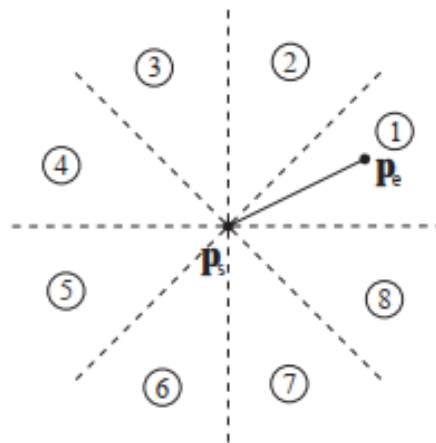


Figure The eight octants with an example line in the first octant

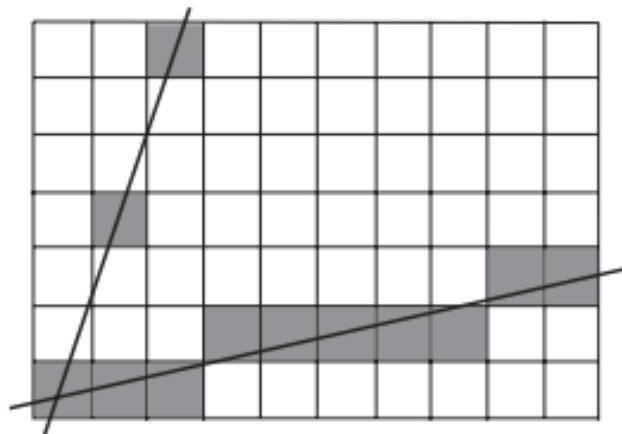


Figure Using the **line1** algorithm in the first and second octants.

The line1 algorithm selects the above pixel sequence:

```
line1 ( int xs, int ys, int xe, int ye, color c )
float s; int x,y; {
s=(ye-ys) / (xe-xs);
(x,y)=(xs,ys);
while (x <= xe) {
setpixel(x,y,c);
x=x+1;
y=ys + round(s * (x-xs));
}
}
```

The while loop is based on the variable of the *major axis* of the line (x). The major axis is x if $|x_e - x_s| > |y_e - y_s|$, otherwise it is y . The non-major axis is called the *minor axis*. If the line1 algorithm is used to draw a line whose major axis is y , then gaps appear (Figure). Instead, a variant which runs the while loop on the y variable should be used in that case. Also note that we should check for the condition $x_e - x_s = 0$ to avoid a division by 0; line rasterization becomes trivial in this case. The value being rounded is increased by s at every iteration of the loop. The expensive round operation can be avoided if we split the y value into an integer and a float part e and compute its value incrementally. The line2 algorithm does this:

```
line2 ( int xs, int ys, int xe, int ye, color c )
float s,e; int x,y;
e=0;
s=(ye-ys) / (xe-xs);
```

```

(x,y)=(xs,ys);
while (x <= xe) {
/* assert  $-1/2 \leq e < 1/2$  */
setpixel(x,y,c);
x=x+1;
e=e+s;
if (e >= 1/2) {
y=y+1;
e=e-1;
}
}
}

```

Notice how the **line2** algorithm resembles the leap-year calculation. The slope is added to the e variable at each iteration until it makes up more than half a unit, and then the line leaps up by 1; the integer y variable is incremented and e is correspondingly reduced, so that the sum of the two variables is unchanged. In a similar manner, there are approximately 365.25 days per year, but calendars are designed with an integer number of days. Hence we add a day every fourth year to make up for the error being accumulated [Harr04]. With suitable scaling, the floating point variables in **line2** can be replaced by integer variables. Multiplying the leap-decision variables by $dx = x_e - x_s$ makes s and e integers. The leap decision becomes $e \geq dx/2$, but since e is now integer, we can replace $dx/2$ by the integer value $dx/2$, which can be computed by a numerical shift without changing the algorithm semantics. We can also replace the test $e \geq dx/2$ by $e \geq 0$ (which is more efficient) using an initial subtraction of $dx/2$ from e . We thus arrive at the Bresenham algorithm [Bres65]:

```
line3 ( int xs, int ys, int xe, int ye, color c )  
int x,y,e,dx,dy;  
e=-(dx >> 1);  
dx=(xe-xs);  
dy=(ye-ys);  
(x,y)=(xs,ys);  
while (x <= xe) {  
/* assert -dx <= e < 0 */  
setpixel(x,y,c);  
x=x+1;  
e=e+dy;  
if (e >= 0) {  
y=y+1;  
e=e-dx;  
}  
}  
}
```

where >> stands for the right shift integer operator (right shifting by 1 bit is equivalent to dividing by 2 and taking the floor). The algorithm line3 is suitable for lines in the first octant. The major axis for each of the eight octants and the action on the variable of the minor axis are given in Table.

Octant	Major axis	Minor axis variable
1	x	increasing
2	y	increasing
3	y	decreasing
4	x	increasing
5	x	decreasing
6	y	decreasing
7	y	increasing
8	x	decreasing

Table Line-rasterization requirements per octant.

EXPERIMENT NO : 1 BRESENHAM'S LINE ALGORITHM

Aim : To write a C program to draw a line using Bresenham's Algorithm.

ALGORITHM :

Step 1 : **Start.**

Step 2 : **Initialize the graphics header files and functions.**

Step 3 : **Declare the required variables and functions.**

Step 4 : **Get the four points for drawing a line namely x_1, x_2, y_1, y_2 .**

Step 5 : **Draw the line using the algorithm.**

Step 6 : **Display the output.**

Step 7 : **stop.**

PROGRAM : else

```
{
#include "stdio.h" x=xa;
#include "conio.h" y=ya;
#include "math.h" xend=xb;
#include "graphics.h" }
```



```
main()
{

int gd=DETECT,gm;
int xa,xb,ya,yb;
int dx,dy,x,y,xend,p;
initgraph(&gd,&gm,"c:\\tc\\bgi");
printf("Enter The Two Left
endpoints(xa,ya):\n");
scanf("%d%d",&xa,&ya);
printf("Enter The Two Right
endpoints(xb,yb):\n");
scanf("%d%d",&xb,&yb);
dx=abs(xa-xb);
dy=abs(ya-yb);
p=2*dy-dx;
if(xa>xb)
{
x=xb;
y=yb;
xend=xa;
}
{
x=xa;
y=ya;
xend=xb;
```

EPNU-23411015
2026/03/13 13:00

```
}  
putpixel(x,y,6);  
while(x<xend)  
  
{  
x=x+1;  
if(p<0)  
{  
p=p+2*dy;  
}  
  
else  
{  
y=y+1;  
p=p+2*(dy-dx);  
}  
  
putpixel(x,y,6);  
}  
getch();  
return(0);  
}
```

EPNU-23411015
2026/03/13 13:00

EXPERIMENT NO : 2 BRESENHAM'S CIRCLE ALGORITHM

Aim : Bresenham's Circle Drawing Algorithm Using C Programming :-

```
#include<stdio.h>

#include<conio.h>

#include<graphics.h>

void main()
{
int gd=DETECT,gm;
int x,y,r;
void cir(int,int,int);
printf("Enter the Mid points and RADIUS:");
scanf("%d%d%d",&x,&y,&r);
initgraph(&gd,&gm,"");
cir(x,y,r);
getch();
closegraph();
}

void cir(int x1,int y1,int r)
{
int x=0,y=r,p=1-r;
void cliplot(int,int,int,int);
cliplot(x1,y1,x,y);
while(x<y)
{
x++;
if(p<0)
```

EPNU-23411015
2026/03/13 13:00

```
p+=2*x+1;
else
{
y--;
p+=2*(x-y)+1;
}
cliplot(x1,y1,x,y);
}
}
void cliplot(int xctr,int yctr,int x,int y)
{
putpixel(xctr +x,yctr +y,1);
putpixel(xctr -x,yctr +y,1);
putpixel(xctr +x,yctr -y,1);
putpixel(xctr -x,yctr -y,1);
putpixel(xctr +y,yctr +x,1);
putpixel(xctr -y,yctr +x,1);
putpixel(xctr +y,yctr -x,1);
putpixel(xctr -y,yctr -x,1);
getch();
}
```

EXPERIMENT NO : 3 – POLYGON FILLING ALGORITHM

3A :-- Aim : To implement FloodFill Algorithm Using C Programming :--

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
#include<graphics.h>

void flood_fill(int x, int y, int ncolor, int ocolor)
{
    if (getpixel(x, y) == ocolor) {
        putpixel(x, y, ncolor);
        flood_fill(x + 1, y, ncolor, ocolor);
        flood_fill(x + 1, y - 1, ncolor, ocolor);
        flood_fill(x + 1, y + 1, ncolor, ocolor);
        flood_fill(x, y - 1, ncolor, ocolor);
        flood_fill(x, y + 1, ncolor, ocolor);
        flood_fill(x - 1, y, ncolor, ocolor);
        flood_fill(x - 1, y - 1, ncolor, ocolor);
        flood_fill(x - 1, y + 1, ncolor, ocolor);
    }
}

void main()
{
    int x, y, ncolor, ocolor;

    clrscr();

    printf("Enter the seed point (x,y): ");
    scanf("%d%d", &x, &y);

    printf("Enter old color : ");
    scanf("%d", &ocolor);

    printf("Enter new color : ");
    scanf("%d", &ncolor);

    int gd = DETECT, gm = DETECT;
```

```
initgraph(&gd, &gm, "");  
cleardevice();  
circle(300, 200, 50);  
flood_fill(x, y, ncolor, ocolor);  
getch();  
}
```

Aim : To implement BoundaryFill Algorithm Using C Programming :-

```
#include<stdio.h>  
#include<conio.h>  
#include<graphics.h>  
void boundary_fill(int x, int y, int fcolor, int bcolor)  
{  
if ((getpixel(x, y) != fcolor) && (getpixel(x, y) != bcolor)) {  
putpixel(x, y, fcolor);  
boundary_fill(x + 1, y, fcolor, bcolor);  
boundary_fill(x - 1, y, fcolor, bcolor);  
boundary_fill(x, y - 1, fcolor, bcolor);  
boundary_fill(x, y + 1, fcolor, bcolor);  
boundary_fill(x + 1, y - 1, fcolor, bcolor);  
boundary_fill(x + 1, y + 1, fcolor, bcolor);  
boundary_fill(x - 1, y - 1, fcolor, bcolor);  
boundary_fill(x - 1, y + 1, fcolor, bcolor);  
}  
}  
void main()
```

```
{  
int x, y, fcolor, bcolor;  
clrscr();  
printf("Enter the seed point (x,y) : ");  
scanf("%d%d", &x, &y);  
printf("Enter boundary color : ");  
scanf("%d", &bcolor);  
printf("Enter new color : ");  
scanf("%d", &fcolor);  
int gd = DETECT, gm = DETECT;  
initgraph(&gd, &gm, "");  
cleardevice();  
boundary_fill(x, y, fcolor, bcolor);  
getch();  
}
```

EPNU-23411015
2026/03/13 13:00

EXPERIMENT NO : 4 – TRANSFORMATION ALGORITHMS

4A :- Aim : To implement 2D Translation Program Using C Programming

```
#include<stdio.h>

#include<conio.h>

#include<graphics.h>

#include<process.h>

#include<math.h>

int x1,y1,x2,y2,x3,y3,mx,my;

void draw();

void tri();

void main()

{

int gd=DETECT,gm; int c;

initgraph(&gd,&gm,"d:\\tc\\bgi ");

printf("Enter the 1st point for the

triangle:"); scanf("%d%d",&x1,&y1);

printf("Enter the 2nd point for the

triangle:"); scanf("%d%d",&x2,&y2);

printf("Enter the 3rd point for the

triangle:"); scanf("%d%d",&x3,&y3);

cleardevice();

draw();

getch();

tri();

getch();

}
```



```
void draw()
{
    line(x1,y1,x2,y2);
    line(x2,y2,x3,y3);
    line(x3,y3,x1,y1);
}

void tri()
{
    int x,y,a1,a2,a3,b1,b2,b3;
    printf("Enter the Transaction
coordinates"); scanf("%d%d",&x,&y);
    cleardevice();
    a1=x1+x;
    b1=y1+y;
    a2=x2+x;
    b2=y2+y;
    a3=x3+x;
    b3=y3+y;
    line(a1,b1,a2,b2);
    line(a2,b2,a3,b3);
    line(a3,b3,a1,b1);
}
```

EPNU-23411015
2026/03/13 13:00

Aim : To implement 2D Scaling Program Using C Programming

```
#include<stdio.h>

#include<conio.h>

#include<graphics.h>

#include<process.h>

#include<math.h>

int x1,y1,x2,y2,x3,y3,mx,my;

void draw();

void scale();

void main()

{

int gd=DETECT,gm;

int c;

initgraph(&gd,&gm," ");

printf("Enter the 1st point for the triangle:");

scanf("%d%d",&x1,&y1);

printf("Enter the 2nd point for the triangle:");

scanf("%d%d",&x2,&y2);

printf("Enter the 3rd point for the triangle:");

scanf("%d%d",&x3,&y3);

draw();

scale();

}

void draw()

{

line(x1,y1,x2,y2);
```

```
line(x2,y2,x3,y3);
line(x3,y3,x1,y1);
}

void scale()
{
int x,y,a1,a2,a3,b1,b2,b3;
int mx,my;
printf("Enter the scalling coordinates");
scanf("%d%d",&x,&y);
mx=(x1+x2+x3)/3; my=(y1+y2+y3)/3;
cleardevice();
a1=mx+(x1-mx)*x;
b1=my+(y1-my)*y;
a2=mx+(x2-mx)*x;
b2=my+(y2-my)*y;
a3=mx+(x3-mx)*x;
b3=my+(y3-my)*y;
line(a1,b1,a2,b2);
line(a2,b2,a3,b3);
line(a3,b3,a1,b1);
draw();
getch();
}
```

EPNU-23411015
2026/03/13 13:00

Aim : To implement 2D Rotation Program Using C Programming

```
#include<stdio.h>
#include<conio.h>
#include<graphics.h>
#include<process.h>
#include<math.h>
void TriAngle(int x1,int y1,int x2,int y2,int x3,int y3);
void Rotate(int x1,int y1,int x2,int y2,int x3,int y3); void
main()
{
int gd=DETECT,gm; int
x1,y1,x2,y2,x3,y3;
initgraph(&gd,&gm," ");
printf("Enter the 1st point for the triangle:");
scanf("%d%d",&x1,&y1);
printf("Enter the 2nd point for the triangle:");
scanf("%d%d",&x2,&y2);
printf("Enter the 3rd point for the triangle:");
scanf("%d%d",&x3,&y3);
TriAngle(x1,y1,x2,y2,x3,y3);
getch();
cleardevice();
Rotate(x1,y1,x2,y2,x3,y3);
setcolor(1);
TriAngle(x1,y1,x2,y2,x3,y3);
getch();
```

```
}  
  
void TriAngle(int x1,int y1,int x2,int y2,int x3,int y3)  
{  
    line(x1,y1,x2,y2);  
    line(x2,y2,x3,y3);  
    line(x3,y3,x1,y1);  
}  
  
void Rotate(int x1,int y1,int x2,int y2,int x3,int y3)  
{  
    int x,y,a1,b1,a2,b2,a3,b3,p=x2,q=y2;  
    float Angle;  
    printf("Enter the angle for rotation:");  
    scanf("%f",&Angle);  
    cleardevice();  
    Angle=(Angle*3.14)/180;  
    a1=p+(x1-p)*cos(Angle)-(y1-q)*sin(Angle);  
    b1=q+(x1-p)*sin(Angle)+(y1-q)*cos(Angle);  
    a2=p+(x2-p)*cos(Angle)-(y2-q)*sin(Angle);  
    b2=q+(x2-p)*sin(Angle)+(y2-q)*cos(Angle);  
    a3=p+(x3-p)*cos(Angle)-(y3-q)*sin(Angle);  
    b3=q+(x3-p)*sin(Angle)+(y3-q)*cos(Angle);  
    printf("Rotate"); TriAngle(a1,b1,a2,b2,a3,b3);  
}
```

EPNU-23411015
2026/03/13 13:00

EXPERIMENT NO : 5 – CLIPPING ALGORITHMS**Aim :– Implementation of Cohen–Sutherland Line Clipping Algorithm**

```
#include<stdio.h>

#include<conio.h>

#include<graphics.h>

#include<math.h>

void clip(float,float,float);

int i,j=0,n;

int rx1,rx2,ry1,ry2;

float x1[8],y1[8];

void main()

{

int gd=DETECT,gm;

int i,n;

float x[8],y[8],m;

clrscr();

initgraph(&gd,&gm,"");

printf("coordinates for rectangle : ");

scanf("%d%d%d%d",&rx1,&ry1,&rx2,&ry2);

printf("no. of sides for polygon : ");

scanf("%d",&n);

printf("coordinates : ");

for(i=0;i<n;i++)

{

scanf("%f%f",&x[i],&y[i]);

}

}
```

```
cleardevice();
outtextxy(10,10,"Before clipping");
outtextxy(10,470,"Press any key....");
rectangle(rx1,ry1,rx2,ry2);
for(i=0;i<n-1;i++)
line(x[i],y[i],x[i+1],y[i+1]);
line(x[i],y[i],x[0],y[0]);
getch();
cleardevice();
for(i=0;i<n-1;i++)
{
m=(y[i+1]-y[i])/(x[i+1]-x[i]);
clip(x[i],y[i],m);
}
clip(x[0],y[0],m);
outtextxy(10,10,"After clipping");
outtextxy(10,470,"Press any key....");
rectangle(rx1,ry1,rx2,ry2);
for(i=0;i<j-1;i++)
line(x1[i],y1[i],x1[i+1],y1[i+1]);
getch();
}
void clip(float e,float f,float m)
{
while(e<rx1 e>rx2 f<ry1 f>ry2)
{
```

EPNU - 23471015
2026/03/13 13:00

```
if(e<rx1)
{
f+=m*(rx1-e);
e=rx1;
}
else if(e>rx2)
{
f+=m*(rx2-e);
e=rx1;
}
if(f<ry1)
{
e+=(ry1-f)/m;
f=ry1;
}
else if(f>ry2)
{
e+=(ry2-f)/m;
f=ry2;
}
x1[j]=e;
y1[j]=f;
j++;
}
}
```

EPNU-23411015
2026/03/13 13:00

References:

1. Fundamentals of Computer Graphics, Shirley, Peter, Ashikhmin, Michael, Marschner, Steve, 2002
2. Jonas Gomes, Luiz Velho, and Mario Costa Sousa, Computer Graphics: Theory and Practice, 2011

EPNU-23411015
2026/03/13 13:00